



GRADO EN ESTADÍSTICAS

TRABAJO FIN DE ESTUDIOS

Programación
Lineal
Entera

Julia López Sacramento

Sevilla, Febrero de 2024

Índice

Prólogo	i
Resumen	ii
Abstract	iii
1 Introducción a la Programación Lineal	1
1.1 Programación Lineal Entera	2
1.2 Clasificación de los Problemas Lineales Enteros	5
2 Modelación	7
2.1 Formulación	8
2.2 Elementos de un Problema de Programación Matemática	10
3 Métodos de resolución de problemas de programación entera	13
3.1 Método del Simplex	13
3.1.1 El algoritmo del Simplex. Forma tabular	15
3.1.1.1 Soluciones básicas factibles	15
3.1.1.2 Condición necesaria y suficiente de optimalidad	19
3.1.1.3 El algoritmo del Simplex	20
3.1.1.4 Simplex en forma tabular	26
3.2 Algoritmos de Ramificación y Acotación	31
3.2.1 Algoritmo de Dakin	31
3.2.2 Programación Booleana	44
3.3 Dual del Simplex y Simplex Columna	50
3.3.1 Dual del Simplex	50
3.3.1.1 Introducción	50
3.3.1.2 Algoritmo dual del Simplex	51

3.3.2	Simplex Columna	53
3.4	Algoritmos de Corte	56
3.4.1	Corte todo entero de Gomory (f-corte)	57
3.4.2	Corte mixto de Gomory(m-corte)	61
3.4.3	Corte mixto de Gomory (m^* - corte)	65
4	Complejidad algorítmica	67
4.1	Introducción	67
4.2	Problemas de Decisión y las Clases $\mathcal{NP}$ y $\mathcal{P}$	69
4.2.1	La clase $\mathcal{P}$ (Polinómicamente acotado)	71
4.2.2	La clase $\mathcal{NP}$ (No determinista Polinómicamente acotado)	72
4.3	Reducción Polinómica y la Clase $\mathcal{NP}$ - completa	72
4.4	El problema $\mathcal{P} = \mathcal{NP}$	74
4.4.1	Planteamiento del problema	75
4.5	Consecuencias de $\mathcal{P} = \mathcal{NP}$ o $\mathcal{P} \neq \mathcal{NP}$	76
5	Implementación en R	79
5.1	Función simplex() del package boot	79
5.2	Función lp() del package lpSolve	82
5.3	Función Rglpk_solve_LP() de la librería Rglpk	84
	Conclusiones	87
	Bibliografía	89

Prólogo

Quiero expresar mi sincero agradecimiento a Antonio Rufian Lizana, mi tutor, por su orientación y apoyo en la realización de este trabajo sobre Programación Lineal Entera.

Su ayuda fue invaluable en cada paso del proceso, y estoy profundamente agradecida por su tiempo y dedicación.

También quiero agradecer a mi familia por su amor, comprensión y apoyo incondicional.

A mis compañeros de clase y amigos les quiero agradecer sus consejos y momentos de distracción, fueron esenciales para mantener el ánimo en los momentos difíciles.

Este trabajo es el resultado del esfuerzo conjunto de muchas personas, y estoy agradecida por todas las contribuciones que lo hicieron posible.

Resumen

El trabajo aborda la programación lineal entera, una técnica clave en la investigación operativa para resolver problemas de toma de decisiones con restricciones lineales y variables enteras.

La estructura del trabajo se divide en los siguientes capítulos:

En el *Capítulo 1* se explican los fundamentos teóricos de la programación lineal y un poco de historia.

El *Capítulo 2* trata sobre la formulación de problemas de programación lineal entera, destacando la importancia de definir correctamente restricciones y objetivos.

En el *Capítulo 3* se describen diversos métodos de resolución como el método del simplex, el algoritmo de ramificación y acotación y los algoritmos de corte.

En el *Capítulo 4* se analiza la complejidad algorítmica de los problemas de programación lineal, discutiendo las clases $\mathcal{P}$ y $\mathcal{NP}$ y el problema $\mathcal{P} = \mathcal{NP}$.

En el *Capítulo 5* se aborda la implementación práctica de algoritmos de programación lineal en el lenguaje R, utilizando librerías específicas para problemas reales.

Abstract

The work addresses integer linear programming, a key technique in operations research for solving decision-making problems with linear constraints and integer variables.

The structure of the work is divided into the following chapters:

In *Chapter 1*, the theoretical foundations of linear programming are explained, including the formulation of problems with linear constraints and objectives.

Chapter 2 deals with the formulation of integer linear programming problems, highlighting the importance of correctly defining constraints and objectives.

In *Chapter 3*, various solution methods are described, such as the branch and bound algorithm, the simplex method, and cutting plane algorithms.

Chapter 4 analyzes the algorithmic complexity of linear programming problems, discussing the classes $\mathcal{P}$ and $\mathcal{NP}$ and the $\mathcal{P} = \mathcal{NP}$ problem.

Chapter 5 addresses the practical implementation of linear programming algorithms in the R language, using specific libraries for real-world problems.

Capítulo 1

Introducción a la Programación Lineal

La Programación Lineal (PL) es una de las técnicas de modelización y resolución de problemas más utilizadas en la Investigación Operativa. Se refiere a la modelización y resolución de problemas de toma de decisiones, en los que las variables de decisión se relacionan mediante restricciones lineales y la función de evaluación de la decisión, o función objetivo, es lineal en las variables de decisión.

La Programación Lineal estudia el problema de minimizar o maximizar una función lineal en la presencia de desigualdades lineales. Desde que George B. Dantzing desarrolló el método simplex en 1947, la programación lineal se ha utilizado extensamente en el área militar, industrial, gubernamental y de planificación urbana, entre otras. La popularidad de la programación lineal se puede atribuir a muchos factores, incluyendo su habilidad de los usuarios para resolver problemas a gran escala en un intervalo razonable de tiempo mediante el uso del método simplex y de computadoras.

A partir de la Segunda Guerra Mundial, se hizo evidente que era esencial la planificación y coordinación entre varios proyectos, así como el uso eficaz de los recursos disponibles. En junio de 1947 se inició un trabajo intensivo del equipo de la Fuerza Aérea de los EE.UU. conocido como SCOOP (Scientific Computation of Optimum Programs). Como resultado, George B. Dantzing desarrolló el método simplex para el final del verano de 1947. El interés en la programación lineal se difundió rápidamente entre economistas, matemáticos, estadísticos e instituciones gubernamentales. En el verano de 1949 se dictó una conferencia sobre programación lineal bajo el patrocinio de la Comisión Cowles para la Investigación en Economía.

Posteriormente, los trabajos presentados en esa conferencia fueron recopilados por T.C Koopmans, en 1951, en el libro *Activity Analysis of Production and Allocation*.

Desde la creación del método simplex mucha gente ha contribuido al crecimiento de la programación lineal, ya desarrollando su teoría matemática, diseñando códigos y métodos computacionales eficientes, experimentando nuevas aplicaciones, y también utilizando la programación lineal como una herramienta auxiliar para resolver problemas más complejos como son programas discretos, programas no lineales, problemas combinatorios, problemas de programación estocástica y problemas de control óptimo.

1.1 Programación Lineal Entera

Los problemas de variables enteras aparecen en problemas cotidianos con mucha facilidad: diseño de horarios para compañías de transporte, planificación de la producción, localización de centros de servicio... No sólo aparecen ligados a aspectos económicos o industriales, sino también a ámbitos estadísticos (análisis de datos), físicos (determinación de estados mínimos de energía) o criptográficos (diseño de códigos seguros), por citar sólo unos pocos. Las variables enteras aparecen en estos problemas de manera natural a la hora de modelar elementos indivisibles (número de productos a fabricar, plantas industriales a localizar, ciudades a visitar...).

Hasta la década de los noventa, la resolución de problemas enteros estaba muy limitada: el tamaño de los problemas que podían resolverse era bastante pequeño pese a poderse resolver problemas lineales (sin restricciones de integridad) de tamaño mucho mayor. Esto es consecuencia del gran incremento en la dificultad de resolución que supone el (aparentemente simple) hecho de exigir que las variables tomen sus valores sobre un conjunto discreto en lugar de sobre uno continuo. Sin embargo, en los últimos años han podido resolverse problemas bastante más difíciles y grandes gracias a una combinación de:

- mejores modelizaciones de los problemas
- ordenadores más potentes
- mejores programas informáticos de Programación Lineal
- nuevos algoritmos y teorías.

Un problema de programación lineal en el que todas las variables han de ser enteras se dice un *problema de programación lineal entero puro*.

Ejemplo 1.1.1.

$$\begin{aligned} \text{Maximizar: } & 5x_1 + 3x_2 \\ \text{s.a: } & 2x_1 + x_2 \leq 10 \\ & x_1 + 3x_2 \leq 12 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \text{ enteras} \end{aligned}$$

Un problema de programación lineal en el que sólo algunas de las variables han de ser enteras se dice un *problema de programación lineal entero mixto*.

Ejemplo 1.1.2.

$$\begin{aligned} \text{Maximizar: } & 5x_1 + 3x_2 \\ \text{s.a: } & 2x_1 + x_2 \leq 10 \\ & x_1 + 3x_2 \leq 12 \\ & x_1, x_2 \geq 0 \\ & x_1 \text{ entera} \end{aligned}$$

La expresión problema de programación entera (PPE) comprende tanto los problemas enteros puros, como los enteros mixtos.

El problema de programación lineal que se obtiene al omitir las restricciones de integridad de un problema entero se le llama problema entero relajado.

La región factible del problema entero relajado contiene a la región factible del problema entero.

El óptimo de problema entero relajado obtiene mejor o igual valor en la función objetivo que el óptimo del problema entero.

Como la región factible del problema entero es mucho más pequeña que la de su contrapartida continua, cabría esperar que los problemas enteros fuesen mucho más fáciles de resolver. Sin embargo, la realidad es diametralmente opuesta: son bastante más difíciles. Esto es consecuencia del carácter discreto del conjunto de soluciones.

Una primera aproximación a la resolución de los problemas enteros podría ser la de redondear los valores del problema relajado. Pero ese camino no conduce a buenas soluciones y hay que descartarlo.

Ejemplo 1.1.3.

$$\begin{aligned} \text{Max} \quad & 21x_1 + 11x_2 \\ \text{s.a:} \quad & 7x_1 + 4x_2 \leq 13 \\ & x_1, x_2 \text{ enteras.} \end{aligned}$$

La región factible del problema es $\mathbb{F} = \{(0,0)(0,1)(0,2)(0,3)(1,0)(1,1)\}$. Si evaluamos dichos puntos nos da que el óptimo es el punto $(0,3)$ y que $z^* = 33$.

Nota 1.1.1. Obviamente el método de evaluación de los puntos factibles no es normalmente una buena estrategia, pues pueden ser muy numerosos o incluso infinitos.

Así pues, resolvamos el problema relajado para redondear las soluciones.

$$\begin{aligned} \text{Max} \quad & 21x_1 + 11x_2 \\ \text{s.a:} \quad & 7x_1 + 4x_2 \leq 13 \\ & x_i \geq 0, \quad i = 1, 2. \end{aligned}$$



La solución por métodos gráficos es $(\frac{13}{7}, 0)$. Redondeando obtenemos $(2, 0)$, que es un punto infactible y por tanto no puede ser la solución. Si redondeamos por abajo para conseguir la factibilidad nos queda $(1, 0)$, que como sabemos tampoco es óptima, ya que el valor de la función objetivo en ese punto es $21 < 33$. Con lo que hemos probado que al redondear una solución es posible que pierda la factibilidad y si es factible puede estar muy alejada del óptimo.

1.2 Clasificación de los Problemas Lineales Enteros

Atendiendo al tipo de variables:

- **Enteros puros:** son aquellos en que todas las variables únicamente pueden tomar valores enteros. También se distinguen dentro de estos los problemas totalmente enteros como aquellos en que tanto las variables como todos los coeficientes que intervienen en el problema han de ser enteros.
- **Mixtos:** son aquellos en los que hay al mismo tiempo variables continuas y variables que sólo pueden tomar valores enteros.
- **Binarios o booleanos:** las variables sólo pueden tomar los valores cero o uno.

Atendiendo al criterio del tipo de problema:

- **Directo:** Si el problema de decisión involucra variables enteras.
- **Codificado:** Cuando se trata de un problema que contiene además de aspectos cuantitativos, alguna consideración de tipo cualitativos, y por ello para tratar este tipo de aspectos se requiere el uso de variable enteras o binarias.
- **Transformado:** Cuando el problema no incluye variables enteras, pero para ser tratado analíticamente requiere el uso de variable enteras “artificiales”.

Capítulo 2

Modelación

La modelación en la IO puede definirse como el proceso de abstracción del sistema real al modelo cuantitativo. Es la construcción del modelo matemático del sistema real bajo estudio. Involucra desde la definición del sistema real y la determinación de sus fronteras, incluyendo la conceptualización del sistema asumido, hasta llegar a la elaboración del modelo en sí. En general se piensa que la modelación se limita únicamente al diseño del modelo, lo cual es erróneo e incompleto. En cierto sentido, podemos concluir que la IO es una ingeniería modelística.

La modelación es sin duda una combinación de arte y ciencia, aunque se debe reconocer que es mucho más arte que ciencia. De hecho, no puede precisarse un algoritmo o un conjunto de reglas para construir a un buen modelo, por lo que necesariamente la modelación se aprende mediante la práctica. Bastará entonces con afirmar que se ha construido un buen modelo cuando éste sirva para alcanzar los propósitos requeridos.

Este argumento se apoya en la experiencia práctica acumulada por diversos investigadores en el campo de operaciones. A partir de esta experiencia, se han establecido una serie de principios generales que guían la construcción de modelos de manera efectiva.

A continuación se presenta una lista de los principios generales de la modelación:

1. No se debe elaborar un modelo complicado cuando uno simple es suficiente.
2. El problema no debe ajustarse al modelo o método de solución.
3. La fase deductiva de la modelación debe realizarse rigurosamente.
4. Los modelos deben validarse antes de su implantación.

5. Nunca debe pensarse que el modelo es el sistema real.
6. Un modelo nunca debe criticarse por algo para lo que no fue hecho.
7. No considere un modelo como la perfección máxima.
8. Uno de los primeros beneficios de la modelación reside en el desarrollo del modelo.
9. Un modelo es tan bueno o tan malo como la información con la que se trabaja.
10. Los modelos no pueden reemplazar al tomador de decisiones.

2.1 Formulación

La estructura de un problema de programación lineal entera queda expresada en los siguientes términos:

Encontrar los valores $x_1, \dots, x_n$ que

$$\begin{array}{ll}
 \text{Maximicen (o minimicen)} & z = \sum_{j=1}^m c_j x_j \\
 \text{sujeto a} & \sum_{j=1}^m a_{ij} x_j \leq (\geq)(=) b_i, \quad i = 1, \dots, m \\
 & x_j \geq 0 \quad j = 1, \dots, m \\
 & x_j \text{ entero} \quad j \in I
 \end{array}$$

donde I es un subconjunto de $\{1, 2, \dots, n\}$.

Si $I = \{1, 2, \dots, n\}$, el problema recibe el nombre de Problema de Programación Lineal Entera Pura. Si I es un subconjunto propio de $\{1, 2, \dots, n\}$, entonces el problema se denomina Problema de Programación Lineal Entera Mixta. En el primer caso se exige a todas las variables de decisión tomar valores enteros, mientras que en el segundo únicamente se hace este requerimiento para algunas variables de decisión.

Designando por $A = (a_{ij})_{m \times n}$, $X = (x_1, \dots, x_n)'$, $B = (b_1, \dots, b_m)'$ y $C' = (c_1, \dots, c_n)$ un problema de Programación Lineal Entera Pura se puede expresar en forma matricial en los siguientes términos:

Encontrar un vector X de tal forma que se

$$\begin{aligned} &\text{Maximice} \quad z = C'X \\ &\text{sujeto a} \quad AX \leq (\geq)(=)B \\ &\quad \quad \quad X \geq 0 \\ &\quad \quad \quad X \text{ entero} \end{aligned}$$

Y designando por

$$\begin{aligned} A &= (D, F), \quad C' = (C'_1, C'_2), \quad C'_1 = (c_{j_1}, \dots, c_{j_k}), \quad C'_2 = (c_{j_{k+1}}, \dots, c_{j_n}), \\ D &= (a_{ij_I}), \quad i = 1, \dots, m, \quad I = 1, \dots, k, \quad F = (a_{ij_I}), \quad i = 1, \dots, m, \quad I = k+1, \dots, n, \\ X' &= (Y'_1, Y'_2) \quad \text{con} \quad Y'_1 = (x_{j_1}, \dots, x_{j_k}) \quad \text{e} \quad Y'_2 = (x_{j_{k+1}}, \dots, x_{j_n}) \end{aligned}$$

Un problema de programación lineal entera mixta se puede expresar como sigue:

Encontrar un vector X de tal forma que se

$$\begin{aligned} &\text{Maximice} \quad z = C'_1 Y_1 + C'_2 Y_2 \\ &\text{sujeto a} \quad DY_1 + FY_2 \leq (\geq)(=)B \\ &\quad \quad \quad Y_1 \geq 0 \\ &\quad \quad \quad Y_2 \geq 0 \\ &\quad \quad \quad Y_2 \text{ entero} \end{aligned}$$

Un especial interés han recibido los problemas particulares de programación lineal entera en los que las variables de decisión únicamente pueden tomar bien el valor **cero** o bien el valor **uno** que se denominan problemas de programación booleana.

El modelo viene propuesto por:

$$\begin{aligned} &\text{Max.} \quad \sum_{j=1}^m c_j x_j \\ &\text{s.a} \quad \sum_{j=1}^m a_{ij} x_j \leq b_i, \quad \forall i = 1, \dots, m \\ &\quad \quad \quad x_j \in \{0, 1\}, \quad \forall j = 1, \dots, m \end{aligned}$$

2.2 Elementos de un Problema de Programación Matemática

A partir de este caso real, vamos a formular el problema de programación matemática lineal correspondiente, identificando sus elementos.

Ejemplo 2.2.1. *Rulisa fabrica masa para pasteles de tipo I y II. La de tipo I la vende a 5 euros el kilo, gastando 1 euro en ingredientes y 2 en mano de obra. La de tipo II se vende a 3 euros y cuestan 1 euro, tanto los ingredientes como el trabajo. Para hacer las masas se necesitan dos tipos de actividades: amasado y horneado. Rulisa dispone de 18 horas de amasado y 12 de horneado a la semana. La masa de tipo I necesita 2 horas de amasado y 3 de horneado, mientras que la de tipo II, necesita 3 de amasado y 1 de horneado. Si la cantidad de masa que se puede vender es ilimitada, optimizar los beneficios semanales de Rulisa.*

1. **Variables de decisión.** Serán aquellas variables que describen los elementos sobre los que podemos decidir. Su determinación debe dar la solución del problema y su acertada elección simplificará mucho el planteamiento. Su elección no es unívoca y un problema de programación matemática puede estar correctamente planteado con elecciones distintas de las variables de decisión. En nuestro ejemplo se trata de saber la cantidad de masa de tipo I y II que hay que fabricar semanalmente. Por tanto,

- Sea x_1 Kilogramos de masa I a fabricar semanalmente.
- Sea x_2 Kilogramos de masa II a fabricar semanalmente.

2. **Función objetivo.** Es la función que expresa lo que queremos optimizar (maximizar o minimizar). En nuestro caso se trata de maximizar la ganancia. $f(x) = (5 - 1 - 2)x_1 + (3 - 1 - 1)x_2 = 2x_1 + x_2$.

Observación 2.2.1. *Obsérvese que $\text{Max } f(x) = \text{Min } -f(x)$*

3. **Las restricciones.** Si el problema no tuviese restricciones la solución sería siempre infinito. Así que consideramos las restricciones en función de las variables de decisión definidas:

- $2x_1 + 3x_2 \leq 18$. Tiempo máximo de amasado permitido.
- $3x_1 + x_2 \leq 12$. Tiempo máximo de horneado permitido.

4. Restricciones de signo. En nuestro caso

- $x_1 \geq 0$
- $x_2 \geq 0$
- x_1, x_2 enteras

Podemos suponer, sin pérdida de generalidad, que todas las variables son no negativas. Si no lo son, veremos métodos de transformación. Además de estas restricciones de signo, puede haber de integridad u otras, que ya veremos.

El problema queda formulado, como:

$$\begin{array}{ll}\text{Max:} & 2x_1 + x_2 \\ \text{s.a:} & 2x_1 + 3x_2 \leq 18 \\ & 3x_1 + x_2 \leq 12 \\ & x_i \geq 0, \quad i = 1, 2\end{array}$$

Capítulo 3

Métodos de resolución de problemas de programación entera

3.1 Método del Simplex

El método más práctico para obtener una solución óptima en el caso continuo, o sea sin restricciones de integridad, se llama método simplex (debido a G.B.Dantzing) que implica un número muy pequeño de pasos para resolver un problema de programación lineal. La alta eficiencia del método simplex ayuda a popularizar el tema de la programación lineal y hace que las aplicaciones industriales y militares sean prácticas.

El método Simplex es un procedimiento iterativo que permite mejorar la solución de la función objetivo en cada paso. El proceso concluye cuando no es posible continuar mejorando dicho valor, es decir, se ha alcanzado la solución óptima (el mayor o menor valor posible, según el caso, para el que se satisfacen todas las restricciones).

Partiendo del valor de la función objetivo en un punto cualquiera, el procedimiento consiste en buscar otro punto que mejore el valor anterior. Como se verá en el método Gráfico, dichos puntos son los vértices del polígono (o poliedro o polícoro, si el número de variables es mayor de 2) que constituye la región determinada por las restricciones a las que se encuentra sujeto el problema (llamada región factible). La búsqueda se realiza mediante desplazamientos por las aristas del polígono, desde el vértice actual hasta uno adyacente que mejore el valor de la función objetivo. Siempre que exista región factible, como su número de vértices y de aristas es finito, será posible encontrar la solución.

El método Simplex se basa en la siguiente propiedad: si la función objetivo Z no toma su valor óptimo en el vértice A , entonces existe una arista que parte de A y mejora el valor de Z .

Será necesario tener en cuenta que el método Simplex únicamente trabaja con restricciones del problema cuyas inecuaciones sean del tipo " $\leq$ " (menor o igual) y sus coeficientes independientes sean mayores o iguales a 0. Por tanto habrá que estandarizar las restricciones para que cumplan estos requisitos antes de iniciar el algoritmo del Simplex. En caso de que después de éste proceso aparezcan restricciones del tipo " $\geq$ " (mayor o igual) o " $=$ " (igualdad), o no se puedan cambiar, será necesario emplear otros métodos de resolución, siendo el más común el método de las Dos Fases o método de las penalizaciones, M-grande.

Para entender completamente cómo funciona, es crucial comprender los conceptos de *punto extremo* y *dirección extrema*.

Definición 3.1.1. Un conjunto $C \subseteq \mathbb{R}^n$ se dice *convexo* si el segmento que une un par cualesquiera de puntos del conjunto pertenece al conjunto, o sea

$$\text{si } x_1, x_2 \in C, \text{ entonces } [x_1, x_2] \subseteq C.$$

De otra forma,

$$\text{si } x_1, x_2 \in C, \text{ entonces } \lambda x_1 + (1 - \lambda)x_2 \in C, \forall \lambda \in [0, 1].$$

Definición 3.1.2. Sea C un conjunto convexo cualquiera, diremos que $\bar{x} \in C$ es un *punto extremo* $\iff \bar{x} = \lambda x_1 + (1 - \lambda)x_2$, con $x_1, x_2 \in C$ y $\lambda \in [0, 1] \Rightarrow \lambda = 0 \vee \lambda = 1$. O lo que es lo mismo, no existen $x_1, x_2 \in C$, tal que $\bar{x} = \lambda x_1 + (1 - \lambda)x_2$, con $\lambda \in (0, 1)$. Por tanto, no existe una combinación lineal convexa estricta para $\bar{x}$.

Definición 3.1.3. Sea $C \in \mathbb{R}^n$ un conjunto convexo no vacío. Diremos que $d \in \mathbb{R}^n$ es una *dirección* $\iff \forall x \in C$ se verifica que $x + \mu d \in C, \forall \mu > 0$.

Definición 3.1.4. Una dirección $d \in C$ diremos que es una *dirección extrema* $\iff$ dados dos direcciones $d_1, d_2 \in C$, verificando $d = \alpha d_1 + \beta d_2$, con α y $\beta \geq 0 \Rightarrow \alpha = 0 \vee \beta = 0$. O sea, una dirección extrema no se puede poner como combinación lineal estricta de otras direcciones.

3.1.1 El algoritmo del Simplex. Forma tabular

3.1.1.1 Soluciones básicas factibles

Consideremos el siguiente poliedro $\mathbb{P} = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$ con $A \in \mathbb{M}_{m \times n}$, $b \in \mathbb{R}^m$, $(m \leq n)$, $x \geq 0$ y $\text{Rango}(A) = \text{Rango}(A, b) = m$.

Observación 3.1.1. *Suponer que todas las regiones factibles de un problema de programación lineal son de esa forma no supone pérdida de generalidad, como veremos posteriormente.*

Observación 3.1.2. *A partir de ahora, $\mathbb{P}$ sería siempre el que hemos definido anteriormente, salvo que se haga constar lo contrario de forma expresa.*

Definición 3.1.5. *Consideremos el poliedro $\mathbb{P}$. Si podemos reordenar las columnas de la matriz A de forma que $A = (B, N)$, donde B es una matriz invertible $m \times m$, entonces al punto $x = \begin{pmatrix} x_B \\ x_N \end{pmatrix}$, donde $x_B = B^{-1}b$ y $x_N = 0$, se le llama solución básica. Si además se verifica que $x_B \geq 0$, entonces se le nota por solución básica factible.*

Observación 3.1.3. *Nótese que x , definido como antes, es una de las soluciones del sistema de ecuaciones compatible indeterminado $Ax = b$.*

Observación 3.1.4. *Nótese que toda solución básica factible es un punto de $\mathbb{P}$.*

Observación 3.1.5. • B es la **matriz básica**

- N es la **matriz no básica**
- x_B son las **variables básicas**
- x_N ó x_{NB} son las **variables no básicas**

Ejemplo 3.1.1. *Calcular las soluciones básicas factibles de la región determinada por las siguientes inecuaciones.*

$$\begin{aligned} x_1 + x_2 &\leq 6 \\ x_2 &\leq 3 \\ x_i &\geq 0, \quad i = 1, 2. \end{aligned}$$

Lo primero que debemos hacer es convertir las desigualdades en igualdades, para lo que introducimos unas variables no negativas llamadas **variables de holgura** y que son auxiliares para el problema.

- $x_1 + x_2 \leq 6 \Rightarrow x_1 + x_2 + x_3 = 6$.
- $x_2 \leq 3 \Rightarrow x_2 + x_4 = 3$.

Observación 3.1.6. Si las inecuaciones hubiesen sido en $\geq$, las variables de holgura hubiesen entrado con signo negativo, por ejemplo $x_1 + x_2 \geq 6 \Rightarrow x_1 + x_2 - x_3 = 6$.

Observación 3.1.7. Si no se exige la no negatividad de las variables, podemos hacer un cambio de variable, como veremos posteriormente.

Por tanto tenemos el siguiente poliedro escrito en forma matricial.

$$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 6 \\ 3 \end{pmatrix}$$

1. Tomamos de A, columnas que formen una matriz cuadrada invertible (2×2).
2. Consideremos las columnas 1 y 2 (a_1, a_2).

Dado $B = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$, la matriz inversa de B, B^{-1} , es:

$$B^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}$$

Dado $b = \begin{pmatrix} 6 \\ 3 \end{pmatrix}$, para encontrar $x_B = B^{-1}b$, realizamos la multiplicación de matrices:

$$x_B = B^{-1}b = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 6 \\ 3 \end{pmatrix} = \begin{pmatrix} 1 \cdot 6 + (-1) \cdot 3 \\ 0 \cdot 6 + 1 \cdot 3 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$$

Por lo tanto, $x_B = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$.

Por lo que reordenando las columnas del vector queda $\begin{pmatrix} 3 \\ 3 \\ 0 \\ 0 \end{pmatrix}$, ya que nuestro problema tiene 4 variables y x_3, x_4 valen 0, por ser no básicas.

3. Consideremos, ahora, las columnas 1 y 3 (a_1, a_3), pero en este caso B no es invertible.

4. Consideremos, ahora, las columnas 1 y 4 (a_1, a_4).

$$B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow x_B = B^{-1}b = \begin{pmatrix} 6 \\ 3 \end{pmatrix}$$

Por lo que reordenando las columnas del vector queda $\begin{pmatrix} 6 \\ 0 \\ 0 \\ 3 \end{pmatrix}$

5. Consideremos, ahora, las columnas 2 y 3 (a_2, a_3). $B = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix} \Rightarrow$
 $x_B = B^{-1}b = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$

Por lo que reordenando las columnas del vector queda $\begin{pmatrix} 0 \\ 3 \\ 3 \\ 0 \end{pmatrix}$

6. Consideremos, ahora, las columnas 2 y 4 (a_2, a_4). $B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix} \Rightarrow$
 $x_B = B^{-1}b = \begin{pmatrix} 6 \\ -3 \end{pmatrix}$ Como tiene componentes negativas es una solución básica, pero no factible.

7. Consideremos, ahora, las columnas 3 y 4 (a_3, a_4). $B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow$
 $x_B = B^{-1}b = \begin{pmatrix} 6 \\ 3 \end{pmatrix}$ Por lo que reordenando las columnas del vector queda $\begin{pmatrix} 0 \\ 0 \\ 6 \\ 3 \end{pmatrix}$

8. Si a los vectores obtenidos les quitamos las dos últimas componentes (variables de holgura que no forman parte del problema) nos quedan las siguientes soluciones básicas factibles.

$$\begin{pmatrix} 3 \\ 3 \end{pmatrix}, \begin{pmatrix} 6 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 3 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Teorema 3.1.1. $x \in \mathbb{P}$ es solución básica factible $\Leftrightarrow x$ es punto extremo.

Corolario 3.1.1. Dado un poliedro $\mathbb{P}$, existe al menos un punto extremo.

Corolario 3.1.2. *Dado un poliedro $\mathbb{P}$ el número de puntos extremos es finito.*

Observación 3.1.8. *No puede haber más puntos extremos que posibles matrices básicas.*

Definición 3.1.6. *Consideremos el poliedro P . $d \in P$ es una dirección extrema $\iff$ podemos reordenar las columnas de la matriz A , con $A = (B, N)$, donde B es una matriz invertible $m \times m$, de forma que $B^{-1}a_j \leq 0$, para $a_j \in N$, y d es de la forma $d = \begin{pmatrix} -B^{-1}a_j \\ e_j \end{pmatrix}$, donde e_j es un vector de $n - m$ componentes con todos sus elementos igual a 0, excepto el j que vale 1.*

Corolario 3.1.3. *El número de direcciones extremas de un poliedro $\mathbb{P}$ es finito*

Ejemplo 3.1.2. *Calcular las direcciones extremas de la región determinada por las siguientes inecuaciones.*

$$\begin{aligned} -x_1 + x_2 &\leq 2 \\ x_2 &\leq 5 \\ x_i &\geq 0, \quad i = 1, 2. \end{aligned}$$

Al igual que hicimos antes, busquemos la igualdad en las ecuaciones

- $-x_1 + x_2 + x_3 = 2$.
- $x_2 + x_4 = 5$.

Por tanto queda, $A = \begin{pmatrix} -1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$

1. Consideremos, las columnas 1 y 2 (a_1, a_2) .

$$B = \begin{pmatrix} -1 & 1 \\ 0 & 1 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} -1 & 1 \\ 0 & 1 \end{pmatrix}$$

2. Ahora multiplicamos por las columnas no básicas (a_3, a_4) .

$$B^{-1}a_3 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}, B^{-1}a_4 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

B^{-1} cumple la hipótesis, así que, aplicando el teorema nos queda

$$d = \begin{pmatrix} -B^{-1}a_3 \\ e_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

que nos conduce a la siguiente dirección extrema $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Si completamos el resto de posibilidades del ejemplo, se puede comprobar que dicha dirección es la única que existe en esa región.

3.1.1.2 Condición necesaria y suficiente de optimalidad

Consideremos el siguiente problema de programación lineal:

$$\begin{aligned} \text{Min} \quad & c^T x = \sum_{i=1}^n c_i x_i \\ \text{s.a.} \quad & Ax = b, \\ & x \geq 0. \end{aligned}$$

Observación 3.1.9. • El vector c es el **vector de costes**.

- b es el **término independiente o vector de recursos**.
- A es la **matriz de coeficientes tecnológicos**.

Observación 3.1.10. El considerar el problema de minimizar, en vez de maximizar, no se pierde generalidad como hemos visto anteriormente, pues si es de maximizar, basta con cambiar de signo al vector de costos.

Teorema 3.1.2. Consideremos el (PPL) y sean $x_1, \dots, x_k$ y $d_1, \dots, d_l$ sus puntos extremos y direcciones extremas, respectivamente. (PPL) tiene una solución óptima finita $\Leftrightarrow$ se verifica que $c^T d_j \geq 0, \forall j = 1, \dots, l$.

Observación 3.1.11. De la demostración se deduce que la solución de un (PPL) con solución óptima finita es necesariamente un punto extremo (como habíamos comprobado gráficamente).

Observación 3.1.12. Si el problema es de maximizar, en la condición hay que cambiar el vector c de signo, con lo que la condición de optimalidad queda $c^T d_j \leq 0, \forall j = 1, \dots, l$.

Observación 3.1.13. *Aplicando lo que sabemos hasta ahora, ya tenemos un primer procedimiento para resolver los (PPL).*

1. *Determinar todos los puntos extremos del (PPL), que son las soluciones básicas factibles.*
2. *Evaluar los puntos extremos con la función objetivo (sabemos que hay un número finito).*
3. *Quedarnos con el de mejor valor.*

En cualquier caso, veremos procedimientos más efectivos.

3.1.1.3 El algoritmo del Simplex

La idea del algoritmo es:

1. Partir de un punto extremo.
2. Comprobar si es el óptimo. Si lo es STOP.
3. En caso de que no lo sea, encontrar otro punto extremo que mejore el valor de la función objetivo.
4. ir a 2.

Sea $\bar{x}$ un punto extremo que, como sabemos, es una solución básica factible. Por tanto, podemos descomponer la matriz $A = (B, N)$, con $B_{m \times m}$ invertible. Esto implica, por ser factible, que:

$$A\bar{x} = b \Rightarrow (B, N) \begin{pmatrix} \bar{x}_b \\ \bar{x}_N \end{pmatrix} = b \Rightarrow B\bar{x}_b + N\bar{x}_N = b \Rightarrow \bar{x}_b = B^{-1}b = \bar{b},$$

Pues $\bar{x}_N = 0$, por ser variables no básicas. En este punto nos preguntamos ¿Es $\bar{x}$ el óptimo? Para ello, lo comparamos con un punto x cualquiera factible. Por ser x factible, se verifica que

- $Ax = b$
- $x \geq 0$

Vamos a calcular el valor de la función objetivo en $\bar{x}$ y x .

$$c^t \bar{x} = (c_B^t, c_N^t) \begin{pmatrix} \bar{x}_B \\ \bar{x}_N \end{pmatrix} = c_B^t \bar{x}_B + c_N^t \bar{x}_N = c_B^t \bar{x}_B.$$

Observación 3.1.14. *Recordemos que para que el producto matricial no varíe debemos reordenar todos los elementos de las matrices.*

Por otro lado, reordenamos el vector x de la misma manera que $\bar{x}$, aunque ahora no podemos afirmar que $x_N = 0$.

$$c^t \bar{x} = (c_B^t, c_N^t) \begin{pmatrix} x_B \\ x_N \end{pmatrix} = c_B^t x_B + c_N^t x_N.$$

Por ser factible tenemos que $Ax = b \Rightarrow$

$$(B, N) \begin{pmatrix} x_B \\ x_N \end{pmatrix} = b \Rightarrow Bx_B + Nx_N = b \Rightarrow Bx_B = b - Nx_N \Rightarrow x_B = B^{-1}b - B^{-1}Nx_N$$

Como $\bar{x}_B = B^{-1}b$, entonces tenemos que

$$x_B = \bar{x}_B - B^{-1}Nx_N$$

Por tanto la función objetivo en x vale

$$\begin{aligned} c^t x &= c_B^t (\bar{x}_B - B^{-1}Nx_N) + c_N^t x_N = c_B^t \bar{x}_B - c_B^t B^{-1}Nx_N + c_N^t x_N = \\ &= c_B^t \bar{x}_B + x_N (-c_B^t B^{-1}N + c_N^t) = c^t \bar{x} - (c_B^t B^{-1}N - c_N^t)x_N \end{aligned}$$

Lo que nos permite comparar los valores de las funciones objetivo.

Así pues tenemos que

$$c^t \bar{x} \leq c^t x \iff (c_B^t B^{-1}N - c_N^t)x_N \leq 0$$

o lo que es lo mismo, ya que $x_N \geq 0$,

$$c_B^t B^{-1}N - c_N^t \leq 0$$

que es la condición de optimalidad de $\bar{x}$.

Observación 3.1.15. Obsérvese que la condición de optimalidad no depende de x . Por tanto, dado un punto extremo cualquiera, se puede saber si es el óptimo o no directamente, sin compararlo con los restantes.

Observación 3.1.16. Al vector $c_B^t B^{-1} B - c_N^t$ se le denota como $z_j - c_j$ y es un vector de $n-m$ componentes. Se le llama **vector de costes reducidos o precios sombra**.

Supongamos ahora que nuestro punto extremo $\bar{x}$ no es óptimo, pues no se verifica la condición de optimalidad y tenemos que el vector de costes reducidos $c_B^t B^{-1} N - c_N^t \not\leq 0$, o sea que $\exists j$, tal que $c_B^t B^{-1} a_j - c_j > 0$, ó lo que es equivalente $c_j - c_B^t B^{-1} a_j < 0$.

Observación 3.1.17. Obsérvese que ahora j determina una componente específica del vector $z_j - c_j$.

Observación 3.1.18. Obsérvese que puede haber otras componentes que verifiquen lo mismo además de la j . La estrategia, ahora pasa por construir un nuevo punto x que verifique:

1. Sea factible.
2. Tenga mejor valor en la función objetivo.
3. Sea también extremo.

Construimos $x = \bar{x} + \lambda d_j$, donde $d_j = \begin{pmatrix} -B^{-1}a_j \\ e_j \end{pmatrix}$ y $\lambda \in \mathbb{R}$, verificando $\lambda > 0$.

1. Para ello hay que probar

(a) $Ax = b$

$$Ax = A(\bar{x} + \lambda d_j) = A\bar{x} + \lambda A d_j = b + \lambda(B, N) \begin{pmatrix} -B^{-1}a_j \\ e_j \end{pmatrix} = b + \lambda(-a_j + a_j) = b$$

- (b) $x \geq 0$. Lo veremos posteriormente.

- 2.

$$\begin{aligned} c^t x &= c^t(\bar{x} + \lambda d_j) = c^t \bar{x} + \lambda c^t d_j = c^t \bar{x} + \lambda(c_B^t, c_N^t) \begin{pmatrix} -B^{-1}a_j \\ e_j \end{pmatrix} \\ &= c^t \bar{x} + \lambda(-c_B^t B^{-1} a_j + c_j) \end{aligned}$$

lo que implica que $c^t x < c^t \bar{x}$, cuando $c_B^t B^{-1} a_j - c_j = z_j - c_j > 0$, que es la hipótesis de partida.

Observación 3.1.19. Obsérvese que la diferencia entre $c^t x$ y $c^t \bar{x}$ es $\lambda(c_B^t B^{-1} a_j - c_j)$, lo que será de utilidad a la hora de escoger la componente que entra en la base, como posteriormente veremos.

Observación 3.1.20. Obsérvese que si $-c_B^t B^{-1} a_j + c_j = 0$, x y $\bar{x}$ tienen el mismo valor en la función objetivo, lo que implica que tenemos una solución múltiple, como posteriormente veremos.

Nos quedaba pendiente probar que $x \geq 0$, lo que haremos a la vez que el apartado 3.

3. Llamemos y_j al vector de m componentes $B^{-1} a_j = (y_{1j}, \dots, y_{ij}, \dots, y_{mj})$ y hagamos disyunción de casos.

- (a) Supongamos que el vector $y_j \leq 0$. Si $y_j \leq 0 \Rightarrow d_j \geq 0 \Rightarrow x \geq 0$ ya que $x = \bar{x} + \lambda d_j$ y todas las componentes del vector son positivas. En este caso el problema no tiene solución acotada, pues el valor de la función objetivo en x decrece cuanto queramos, sin más que aumentar λ . De hecho, d_j es una dirección extrema y se verifica que:

$$c^t d_j = (c_B^t, c_N^t) \begin{pmatrix} -B^{-1} a_j \\ e_j \end{pmatrix} = -c_B^t B^{-1} a_j + c_j < 0,$$

por lo que el problema es no acotado sin más que aplicar el teorema 2 de optimalidad de la Programación Lineal.

- (b) Supongamos ahora que el vector $y_j \not\leq 0$, lo que significa que alguna de sus componentes es estrictamente mayor que 0. La estrategia es ahora escoger un λ adecuado para que x sea mayor que 0, lo cual no es difícil haciendo λ suficientemente pequeño. Pero lo que haremos será determinarlo de forma que $x \geq 0$ y además sea un punto extremo. Sea por tanto

$$\lambda = \min_{1 \leq i \leq m} \left\{ \frac{\bar{b}_i}{y_{ij}}, \text{ tal que } y_{ij} > 0 \right\}$$

Observación 3.1.21. Recuérdese que $\bar{b} = B^{-1} b = x_B$.

Observación 3.1.22. λ está bien definido porque por hipótesis $y_j \not\leq 0$, lo que significa que al menos una de sus componentes es estrictamente mayor que 0.

Como el conjunto definido es finito, entonces $\exists r \in \{1, \dots, m\}$ tal que $\lambda = \bar{b}_r / y_{rj}$. Veamos que con ese λ , se verifica que $x \geq 0$.

- Para las componentes básicas de x , o sea, las m primeras (excepto x_r), tenemos

$$x_i = \bar{b}_i + \frac{\bar{b}_r}{y_{rj}}(-y_{ij}).$$

1. En el caso de que $y_{ij} \leq 0 \Rightarrow x_i \geq 0$, obviamente.
2. Si $y_{ij} > 0$. Entonces

$$x_i \geq 0 \Leftrightarrow \bar{b}_i - \frac{\bar{b}_r}{y_{rj}}y_{ij} \geq 0 \Leftrightarrow \frac{\bar{b}_i}{y_{ij}} \geq \frac{\bar{b}_r}{y_{rj}},$$

pero eso es cierto por la definición de λ . Por lo que en cualquiera de los dos casos $x_i \geq 0$ en las componentes básicas.

- Veamos como queda la componente básica x_r .

$$x_r = \bar{b}_r - \frac{\bar{b}_r}{y_{rj}}y_{rj} = \bar{b}_r - \bar{b}_r = 0,$$

por lo que sale de la base.

- La componente no básica x_j queda

$$x_j = 0 + \lambda = \frac{\bar{b}_r}{y_{rj}}$$

y entra en la base.

- El resto de las no básicas valen 0, por la definición de x .

Así pues hemos probado que $x \geq 0$ en cualquier caso. Además x es un punto extremo, porque si a la matriz B de $\bar{x}$ le sustituimos la columna a_r por a_j , las nuevas columnas son linealmente independientes y por tanto forman una nueva base. Así pues como hemos cambiado de base, hemos pasado de un punto extremo a otro.

En estas condiciones ya podemos enunciar **El algoritmo del Simplex (Dantzing, 1947-2005)**.

Consideremos el problema (PPL)

1. Encontrar un punto extremo $\bar{x}$ con una base B .
2. Calcular el vector de costes reducidos $z_j - c_j = c_B^t B^{-1} N - c_N^t$

- (a) Si $z_j - c_j \leq 0$, entonces $\bar{x}$ es el óptimo. STOP
- (b) Si $z_j - c_j \not\leq 0$, sea j tal que $c_B^t B^{-1} a_j - c_j = \text{Max}\{c_B^t B^{-1} N - c_N^t\}$, o sea x_j es la componente que peor cumple la condición de optimalidad. Por tanto, x_j entra en la base.
 - i. Si $y_j = B^{-1} a_j \leq 0$, entonces se tiene una solución ilimitada en la dirección

$$d_j = \begin{pmatrix} -B^{-1} a_j \\ e_j \end{pmatrix} \text{ con el punto } x = \bar{x} + \mu d_j, \mu > 0.$$

- ii. Si $y_j \not\leq 0$, entonces ir a paso 3.

3. Calcular la componente r básica que verifica que

$$\lambda = \min_{1 \leq i \leq m} \left\{ \frac{\bar{b}_i}{y_{ij}} \text{ tal que } y_{ij} > 0 \right\} = \frac{\bar{b}_r}{y_{rj}}.$$

Por tanto, x_r sale de la base. Pivotamos y calculamos el nuevo punto extremo con la nueva base.

$$x_i = \bar{b}_i - \frac{\bar{b}_r}{y_{rj}} y_{ij} \quad i = 1, \dots, m \text{ con } i \neq r$$

$$x_r = 0 \text{ sale de la base}$$

$$x_j = \frac{\bar{b}_r}{y_{rj}} \text{ y entra en la base}$$

4. Formar una nueva base B , sustituyendo a_r por a_j . Ir a 2.

El algoritmo del simplex va de punto extremo en punto extremo, parando cuando encuentra el óptimo. El algoritmo converge porque el número de puntos extremos es finito.

Observación 3.1.23. *El algoritmo sólo necesita que la variable que entre en la base no cumpla la condición de optimalidad. El que escojamos la que peor la cumple es una condición adicional no imprescindible aunque, en general, nos lleva al óptimo en menos iteraciones. El motivo es que la mejora en la función objetivo con el nuevo punto extremo, como ya vimos, viene determinada por la expresión:*

$$\lambda(-c_B^t B^{-1} a_j + c_j) = -\lambda(z_j - c_j),$$

por lo que escogemos el mayor de uno de los componentes de ese producto $(z_j - c_j)$

Observación 3.1.24. En el caso de que una componente no básica $z_j - c_j = 0$, si introducimos la variable x_j en la base es evidente que el valor de la función objetivo no varía. Así pues, cuando estemos en un punto que verifique la condición de optimalidad con una de sus componentes no básicas x_j , verificando $z_j - c_j = 0$, eso significa la existencia de una posible solución múltiple, que encontraremos introduciendo a x_j en la base y procediendo en la forma habitual que exige el algoritmo.

Observación 3.1.25. Una vez escogida la variable que sale de la base, la que entra está determinada por el valor de λ , y sólo puede entrar la variable que verifique la condición exigida. O sea, en la variable de entrada hay cierto margen de discrecionalidad, pero no en la de salida, salvo empates.

3.1.1.4 Simplex en forma tabular

El algoritmo del simplex usado de forma matricial, tal y como hemos visto, es muy laborioso y es fácil cometer errores. La manera más natural de abordarlo es usando tablas, como veremos. Para ello, daremos una tabla general de un (PPL) de minimizar, en la que sin pérdida de generalidad supondremos que las variables básicas son las m primeras.

		x_1	$\cdots$	x_m	x_{m+1}	$\cdots$	x_j	$\cdots$	x_n	
x_1	c_1				$y_{1,m+1}$	$\cdots$	y_{1j}	$\cdots$	y_{1n}	$\bar{b}_1$
$\vdots$	$\vdots$				$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_r	c_r			I	$y_{r,m+1}$	$\cdots$	y_{rj}	$\cdots$	y_{rn}	$\bar{b}_r$
$\vdots$	$\vdots$				$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_m	c_m				$y_{m,m+1}$	$\cdots$	y_{mj}	$\cdots$	y_{mn}	$\bar{b}_m$
		0	$\cdots$	0	$z_{m+1} - c_{m+1}$	$\cdots$	$z_j - c_j$	$\cdots$	$z_n - c_n$	z

Observación 3.1.26. Las variables básicas forman la identidad en la tabla.

Observación 3.1.27. Por z denotamos el valor de la función objetivo.

Observación 3.1.28. Las componentes básicas del vector $z_j - c_j$ valen cero.

Observación 3.1.29. La matriz formada por los vectores $(y_{m+1}, \dots, y_j, \dots, y_n)$ es, en realidad, $B^{-1}N$, pues $y_j = B^{-1}a_j$.

Supongamos que no se verifica la condición de optimalidad. Sea

$$z_j - c_j = \max_{m+1 \leq i \leq n} \{z_i - c_i\}$$

Si $y_j \not\leq 0$, con lo que la solución no es ilimitada, entonces sea

$$\lambda = \min_{1 \leq i \leq m} \left\{ \frac{\bar{b}_i}{y_{ij}}, \text{ tal que } y_{ij} > 0 \right\} = \frac{\bar{b}_r}{y_{rj}},$$

por lo que x_r sale de la base. En este punto pivotamos sobre y_{rj} , que consiste en, mediante transformaciones elementales de filas, conseguir un 1 en la posición y_{rj} , y ceros en los restantes elementos de la columna. Con eso conseguimos recuperar la matriz identidad formada por las variables básicas.

Observación 3.1.30. *El proceso de pivotaje es equivalente a multiplicar la tabla por la inversa de la nueva matriz básica.*

Ejemplo 3.1.3. *Retomemos nuestro problema inicial*

$$\begin{aligned} \text{Max} \quad & 2x_1 + x_2 \\ \text{s.a:} \quad & 2x_1 + 3x_2 \leq 18 \\ & 3x_1 + x_2 \leq 12 \\ & x_i \geq 0, \quad i = 1, 2. \end{aligned}$$

Lo primero es ponerlo en restricciones de igualdad, introduciendo variables de holgura.

$$\begin{aligned} \text{Max} \quad & 2x_1 + x_2 \\ \text{s.a:} \quad & 2x_1 + 3x_2 + x_3 = 18 \\ & 3x_1 + x_2 + x_4 = 12 \\ & x_i \geq 0, \quad i = 1, 2, 3, 4. \end{aligned}$$

En este caso hay un punto extremo inicial muy sencillo, que es el asociado a las variables x_3, x_4 y, por tanto, las columnas a_3 y a_4 , que forman la identidad. Así, la primera tabla queda:

		2	1	0	0	
x_3	0	2	3	1	0	18
x_4	0	3	1	0	1	12
		-2	-1	0	0	0

Observación 3.1.31. *En la primera fila hemos puesto el vector de costes, en vez del nombre de las variables, pues es más operativo.*

Observación 3.1.32. *La ventaja de partir de una matriz básica identidad es que para la primera tabla podemos copiar la matriz A directamente. En otro caso, tendríamos que multiplicarla por B^{-1} .*

La variable que sale de la base se obtiene de calcular el $\min\{-2, -1\}$ que son los valores del vector de costes reducidos que no verifican la condición de optimalidad, recuérdese que el problema es de maximizar, por tanto x_1 entra en la base. La variable que sale de la base se obtiene como $\text{Min}\{\frac{18}{2}, \frac{12}{3}\} = 4$, así pues es x_4 . Pivotamos y obtenemos la siguiente tabla.

		2	1	0	0	
x_3	0	0	7/3	1	-2/3	10
x_1	2	1	1/3	0	1/3	4
		0	-1/3	0	2/3	8

En este caso, la variable que entra en la base es x_2 , pues es la única que no verifica la condición de optimalidad, y la que sale se deduce de $\text{Min}\{\frac{10}{7/3}, \frac{4}{1/3}\} = \text{Min}\{\frac{30}{7}, 12\} = \frac{30}{7}$, por lo que sale x_3 . Y la tabla queda:

		2	1	0	0	
x_2	1	0	1	3/7	-2/7	30/7
x_1	2	1	0	-1/7	3/7	18/7
		0	0	1/7	4/7	66/7

Como se cumple la condición de optimalidad, estamos en el óptimo. La solución es $(\frac{18}{7}, \frac{30}{7})$, con un valor de la función objetivo igual a $\frac{66}{7}$.

Observación 3.1.33. En nuestro caso la base óptima está formada por las columnas (a_2, a_1) .

Observación 3.1.34. Obsérvese que hay que respetar el orden de las columnas de A , tal y como aparecen en la tabla.

Así tenemos que

$$B = \begin{pmatrix} 3 & 2 \\ 1 & 3 \end{pmatrix} \Rightarrow B^{-1} = \begin{pmatrix} 3/7 & -2/7 \\ -1/7 & 3/7 \end{pmatrix}$$

Obsérvese que la tabla final es la matriz A del problema original multiplicada por B^{-1} y eso ocurre con todas las tablas.

Ejemplo 3.1.4.

$$\begin{aligned} \text{Max} \quad & 2x_1 + 3x_2 \\ \text{s.a:} \quad & -x_1 + x_2 \leq 2 \\ & x_2 \leq 5 \\ & x_i \geq 0, \quad i = 1, 2. \end{aligned}$$

Lo primero es ponerlo en restricciones de igualdad, introduciendo variables de holgura.

$$\begin{aligned}
 \text{Max} \quad & 2x_1 + 3x_2 \\
 \text{s.a:} \quad & -x_1 + x_2 + x_3 = 2 \\
 & x_2 + x_4 = 5 \\
 & x_i \geq 0, \quad i = 1, 2, 3, 4.
 \end{aligned}$$

La primera tabla queda:

		2	3	0	0	
x_3	0	-1	1	1	0	2
x_4	0	0	1	0	1	5
		-2	-3	0	0	0

Entra x_2 y sale x_3 .

Observación 3.1.35. En este caso, hemos aplicado la regla del simplex de forma estricta, pero se obtiene la solución de forma más rápida si decidimos que entre x_1 , aunque no sea el que peor cumple la condición de optimalidad, ya que nos sale que el problema es ilimitado en esta primera iteración.

En cualquier caso seguimos de la forma habitual.

		2	3	0	0	
x_2	3	-1	1	1	0	2
x_4	0	1	0	-1	1	3
		-5	0	3	0	6

Entra x_1 y sale x_4

		2	3	0	0	
x_2	3	0	1	0	1	5
x_1	2	1	0	-1	1	3
		0	0	-2	5	21

En este caso debería entrar x_3 , pero $y_3 = B^{-1}a_3 = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \leq 0$. Por tanto, tenemos solución ilimitada.

$x = \bar{x} + \mu d_3$, donde $d_3 = \begin{pmatrix} -y_3 \\ e_3 \end{pmatrix}$ y $\mu \in \mathbb{R}$, verificando $\mu > 0$. Con lo que tenemos

$$\begin{pmatrix} 3 \\ 5 \\ 0 \\ 0 \end{pmatrix} + \mu \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}.$$

Lo que deja al pasarlo a las dos variables del problema y poniendo los vectores de forma horizontal, la solución $(3, 5) + \mu(1, 0)$ con $\mu > 0$.

Ejemplo 3.1.5.

$$\begin{array}{ll} \text{Max} & x_1 + x_2 \\ \text{s.a:} & x_1 + x_2 \leq 4 \\ & -x_1 + x_2 \leq 1 \\ & x_i \geq 0, \quad i = 1, 2. \end{array}$$

Introducimos variables de holgura.

$$\begin{array}{ll} \text{Max} & x_1 + x_2 \\ \text{s.a:} & x_1 + x_2 + x_3 = 4 \\ & -x_1 + x_2 + x_4 = 1 \\ & x_i \geq 0, \quad i = 1, 2, 3, 4. \end{array}$$

La primera tabla queda:

		1	1	0	0	
x_3	0	1	1	1	0	4
x_4	0	-1	1	0	1	1
		-1	-1	0	0	0

Como tenemos empate, pueden entrar tanto x_1 como x_2 . Elegimos x_1 (orden lexicográfico).

Sale x_3

		1	1	0	0	
x_1	1	1	1	1	0	4
x_4	0	0	2	1	1	5
		0	0	1	0	4

Ya estamos en el óptimo, que es $(4, 0)$, pero tenemos coste reducido no básico igual a cero ($z_2 - c_2 = 0$), con lo que hay una posible solución múltiple.

Por ello introducimos a x_2 en la base y sale x_4 .

		1	1	0	0	
x_1	1	1	0	1/2	-1/2	3/2
x_2	1	0	1	1/2	1/2	5/2
		0	0	1	0	4

Así obtenemos otra solución óptima $(\frac{3}{2}, \frac{5}{2})$. Por lo que la solución del problema es

$$\lambda(4, 0) + (1 - \lambda)(\frac{3}{2}, \frac{5}{2})$$

con $\lambda \in [0, 1]$.

Observación 3.1.36. Obsérvese que en la segunda tabla óptima se verifica que el coste reducido no básico $z_4 - c_4 = 0$, lo que es lógico y nos indica que si introducimos x_4 en la base obtenemos otra solución óptima, que sería la que encontramos en primer lugar.

3.2 Algoritmos de Ramificación y Acotación

Una primera técnica de resolución de los problemas enteros, cuya idea es simple, pero que es muy eficaz, consiste en dividir el problema en subproblemas más pequeños y más sencillos de resolver. Después, nos quedaremos con la mejor de las soluciones de esos subproblemas. Son los conocidos como algoritmos de ramificación y acotación (Branch and Bound).

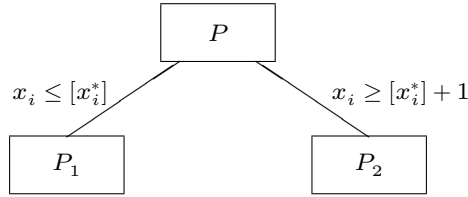
3.2.1 Algoritmo de Dakin

Es uno de los algoritmos B&B y como todos ellos consta de dos procesos:

1. **Ramificación** consiste en la división del problema relajado en dos subproblemas que surgen de la partición del espacio factible continuo del problema relajado en dos subconjuntos también continuos. Si el óptimo del problema relajado en la variable x_i toma el valor $x_i^* \notin \mathbb{Z}$, entonces hacemos la siguiente partición en dos subproblemas (P1,P2), introduciendo nuevas restricciones al problema.

$$X_1 = X \cap \{x : x_i \leq [x_i^*]\} X_2 = X \cap \{x : x_i \geq [x_i^*] + 1\}$$

donde $[x_i^*]$ denota la parte entera de x_i^* . Observar que en esta partición se eliminan solo partes no factibles enteras del problema relajado ($X = X_1 \cup X_2 \cup \{x \in X : x_i \in ([x_i^*], [x_i^*] + 1)\}$)



Seguiremos ramificando hasta encontrar la solución entera óptima del problema original.

2. **Acotación** consiste en operar de forma que no sea necesario sondear explícitamente todas las ramificaciones. Así pues, no será necesario sondear ni ramificar un subproblema cuando se den alguna de las siguientes circunstancias:

- (a) El subproblema es infactible: todos los que surgan de la ramificación de este, serán también infactibles.
- (b) El valor objetivo del óptimo del subproblema (continuo) no es mejor que el valor objetivo de una solución encontrada previamente y que es factible para el problema (está acotado).

Llevando a cabo las acciones anteriores podemos desarrollar el siguiente algoritmo:

Paso 1: Obtener el valor óptimo z_P^* y la solución óptima, x_P^* del problema relajado. Establecer (P_0) , $z^* = z_P^*$, $x^* = x_P^*$ como elementos del nodo raíz n_0 . Tomar $z_{UB} = +\infty$ y $z_{LB} = z^*$.

Paso 2:

2.1) Si $x^* \in Z_n$. **STOP**.

2.2) Si $x^* \notin Z_n$, entonces $\exists x_i^* \notin Z$. Entonces establecer dos ramas que parten del nodo n_0 ,

Nodo n_1 , constituido por el problema (P_1) resultante de añadir al problema (P) la restricción $x_i \leq [x_i^*]$. Sean $(x_1)^*$ la solución óptima y z_1^* el valor óptimo de (P_1) .

Nodo n_2 , constituido por el problema (P_2) resultante de añadir al problema (P) la restricción $x_i \geq [x_i^*] + 1$. Sean $(x_2)^*$ la solución óptima y z_2^* el valor óptimo de (P_2) .

- (a) Si algún (P_i) es infactible esa rama no tiene que seguir siendo sondeada.
- (b) Si $z_i^* > z_{UB}$, esa rama no tiene que seguir siendo sondeada.
- (c) Si $(x_i)^* \notin Z_n$, tomar $z_{LB} = z_i^*$ (en el subárbol que se va a generar) y repetir *Paso 2.2* tomando n_i como n_0 .
- (d) Si $(x_i)^* \in Z_n$ y $z_i^* = z_{LB}$ entonces $(x_i)^*$ es la solución óptima del problema entero. En caso contrario establecer $z_{UB} = z_i^*$

Nota 3.2.1. *El algoritmo B&B termina en un número finito de pasos. Si todos los nodos sondeados ofrecen soluciones no enteras entonces el problema es infactible. En otro caso, la solución (o soluciones) óptima es la correspondiente al nodo tal que $z_i^* = z_{UB}$*

Nota 3.2.2. *El algoritmo B&B se ha expuesto suponiendo que el problema a resolver es un problema entero puro de minimizar. En caso de que se trate de un problema de maximizar y/o entero mixto se reajusta convenientemente.*

Nota 3.2.3. *Se trata de conseguir una enumeración implícita de las soluciones factibles sin necesidad de sondearlas todas explícitamente. Así pues, no será necesario sondear ni ramificar un nodo cuando se den alguna de las siguientes circunstancias:*

1. *El subproblema es infactible.*
2. *El subproblema tiene una solución que satisface las condiciones de integridad del problema original.*
3. *El valor del óptimo del subproblema no es mejor que una solución encontrada previamente y que es factible para el problema original (está acotado).*

Nota 3.2.4. *El algoritmo requiere para ser eficaz que tengamos soluciones enteras para comparar y evitar sondear ramas no prometedoras, por ello debemos utilizar una disciplina LIFO en la resolución de problemas. O sea, resolver primero, los últimos problemas creados.*

Ejemplo 3.2.1. *Rulisa fabrica mesas y sillas. Una mesa necesita 1 hora de trabajo y 9 tablas, y una silla requiere 1 hora de trabajo y 5 tablas. Disponemos de 6 horas de trabajo y tenemos 45 tablas. Sabiendo que el beneficio de construir una mesa es 8 euros y de una silla 5 euros. Determinar la política de fabricación para maximizar el beneficio.*

Sea x_1 = Número de mesas fabricadas.

Sea x_2 = Número de sillas fabricadas.

El problema queda

$$\begin{aligned}
 \text{Max} \quad & 8x_1 + 5x_2 \Leftrightarrow \text{Min} \quad -8x_1 - 5x_2 \\
 \text{s.a:} \quad & x_1 + x_2 \leq 6 \\
 & 9x_1 + 5x_2 \leq 45 \\
 & x_i \geq 0, \quad i = 1, 2 \\
 & x_i \text{ enteras } i = 1, 2.
 \end{aligned}$$

Empezamos por resolver el problema relajado (simplex).

$$\begin{array}{c}
 \begin{array}{cccccc}
 & & -8 & -5 & 0 & 0 \\
 & & x_1 & x_2 & x_3 & x_4 \\
 \begin{array}{|c|c|c|c|c|c|}
 x_3 & 0 & 1 & 1 & 1 & 0 & 6 \\
 x_4 & 0 & \mathbf{9} & 5 & 0 & 1 & 45
 \end{array}
 \end{array}
 \Rightarrow
 \begin{array}{c}
 \begin{array}{cccccc}
 & & -8 & -5 & 0 & 0 \\
 & & x_1 & x_2 & x_3 & x_4 \\
 \begin{array}{|c|c|c|c|c|c|}
 x_3 & 0 & 0 & \mathbf{4/9} & 1 & -1/9 & 1 \\
 x_1 & -8 & 1 & 5/9 & 0 & 1/9 & 5
 \end{array}
 \end{array}
 \end{array}$$

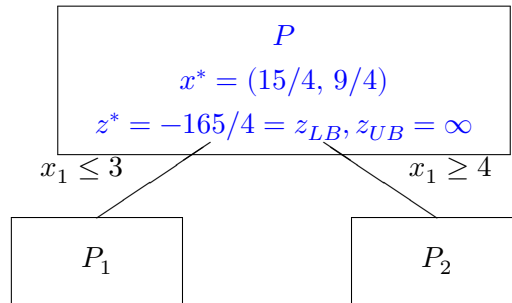
$$\begin{array}{c}
 \begin{array}{cccccc}
 & & -8 & -5 & 0 & 0 \\
 & & x_1 & x_2 & x_3 & x_4 \\
 \begin{array}{|c|c|c|c|c|c|}
 x_2 & -5 & 0 & 1 & 9/4 & -1/4 & 9/4 \\
 x_1 & -8 & 1 & 0 & -5/4 & 1/4 & 15/4
 \end{array}
 \end{array}
 \end{array}$$

La solución óptima es $x_1^* = \frac{15}{4}$ y $x_2^* = \frac{9}{4}$, con $z^* = -\frac{165}{4} = -41.25$. El siguiente paso es escoger una variable con valor fraccionario que tenga que ser entera.

- Nodo n_1 : (P_1). Se añade la restricción $x_1 \leq [x_1^*] \Rightarrow x_1 \leq 3$.
- Nodo n_2 : (P_2). Se añade la restricción $x_1 \geq [x_1^*] + 1 \Rightarrow x_1 \geq 4$.

Nota 3.2.5. Observamos que la unión de las regiones factibles de los subproblemas contiene toda la región factible del problema entero original. (Sólo se han eliminado partes que no contienen puntos enteros).

El árbol queda, por tanto:



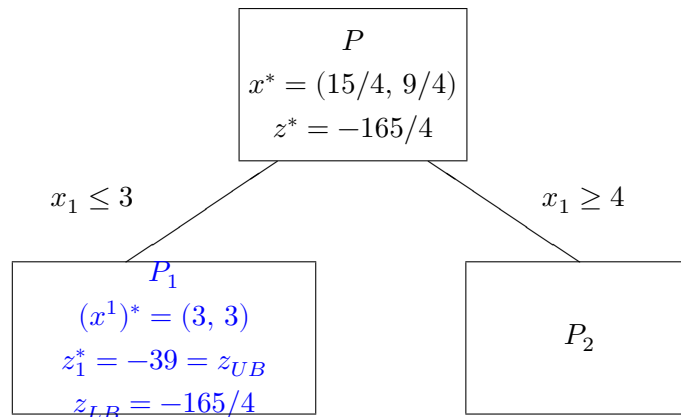
Resolvemos P_1 (simplex).

		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	9/4	-1/4	0	9/4
x_1	-8	1	0	-5/4	1/4	0	15/4
x_5	0	1	0	0	0	1	3
		0	0	-5/4	-3/4	0	-165/4

		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	9/4	-1/4	0	9/4
x_1	-8	1	0	-5/4	1/4	0	15/4
x_5	0	0	0	5/4	-1/4	1	-3/4
		0	0	-5/4	-3/4	0	-165/4

		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	1	0	-1	3
x_1	-8	1	0	0	0	1	3
x_4	0	0	0	-5	1	-4	3
		0	0	-5	0	-3	-39

óptimo (3,3). Como la solución es entera $(x^1)^* \in \mathbb{Z}^2$, se trata de un nodo terminal.



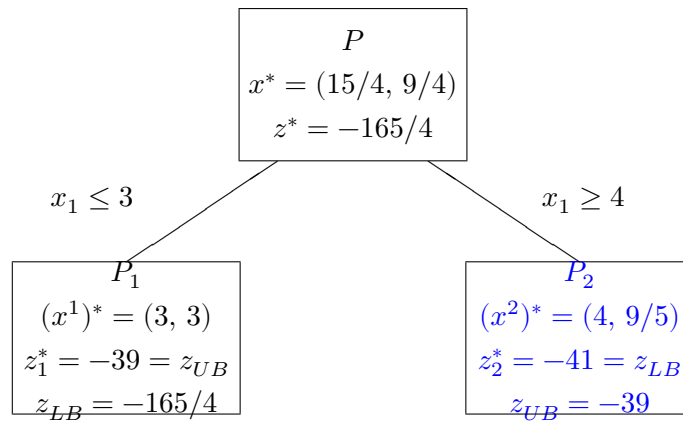
Resolvemos ahora P_2 .

		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	9/4	-1/4	0	9/4
x_1	-8	1	0	-5/4	1/4	0	15/4
x_5	0	-1	0	0	0	1	-4
		0	0	-5/4	-3/4	0	-165/4

		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	9/4	-1/4	0	9/4
x_1	-8	1	0	-5/4	1/4	0	15/4
x_5	0	0	0	-5/4	1/4	1	-1/4
		0	0	-5/4	-3/4	0	-165/4

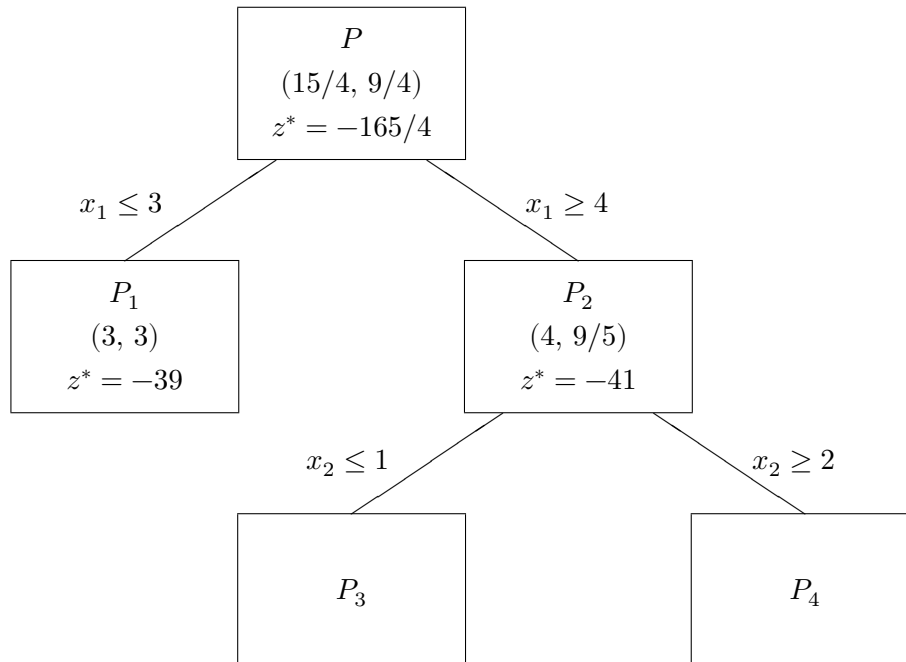
		-8	-5	0	0	0	
		x_1	x_2	x_3	x_4	x_5	
x_2	-5	0	1	0	1/5	9/5	9/5
x_1	-8	1	0	0	0	-1	4
x_3	0	0	0	1	-1/5	-4/5	1/5
		0	0	0	-1	-1	-41

óptimo $(4, \frac{9}{5})$ y valor objetivo -41. Como $(x^2)^* \notin \mathbb{Z}^2$ y además $z_2^* = -41 < -39$ (lo que significa que el nodo no está acotado), tenemos, por tanto, un nodo que no es terminal y que requerirá nuevas ramificaciones.



Hay que seguir ramificando en x_2 , a partir de P_2 .

- Nodo n_1 : (P_3), se añade la restricción $x_2 \leq [x_2^*] \Rightarrow x_2 \leq 1$.
- Nodo n_2 : (P_4), se añade la restricción $x_2 \geq [x_2^*] + 1 \Rightarrow x_2 \geq 2$.



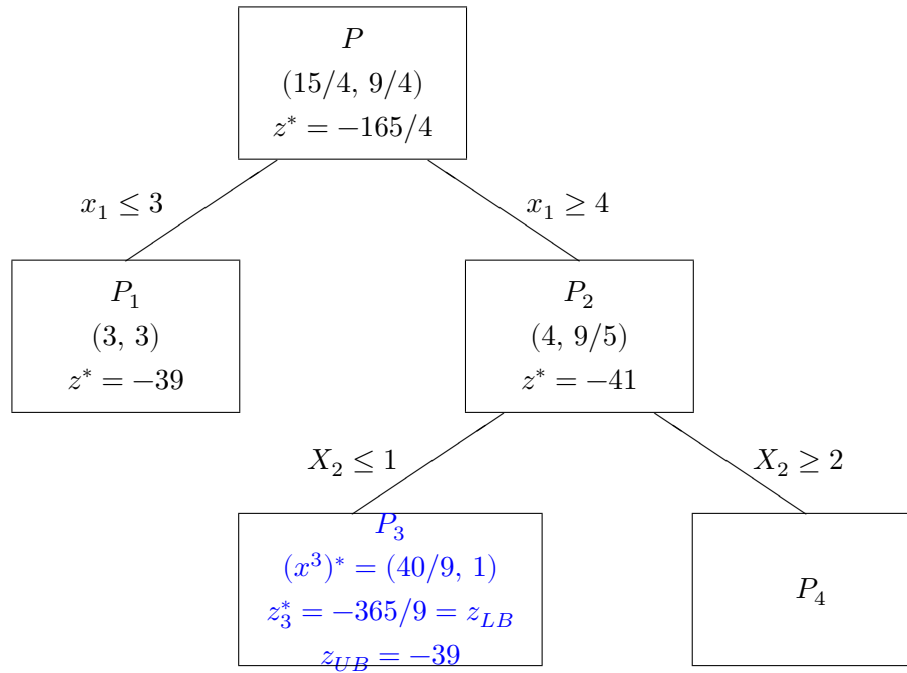
Resolvemos P_3 mediante el simplex, queda:

		-8	-5	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	
x_2	-5	0	1	0	1/5	9/5	0	9/5
x_1	-8	1	0	0	0	-1	0	4
x_3	0	0	0	1	-1/5	-4/5	0	1/5
x_6	0	0	1	0	0	0	1	1
		0	0	0	-1	-1	0	-41

		-8	-5	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	
x_2	-5	0	1	0	1/5	9/5	0	9/5
x_1	-8	1	0	0	0	-1	0	4
x_3	0	0	0	1	-1/5	-4/5	0	1/5
x_6	0	0	0	0	-1/5	-9/5	1	-4/5
		0	0	0	-1	-1	0	-41

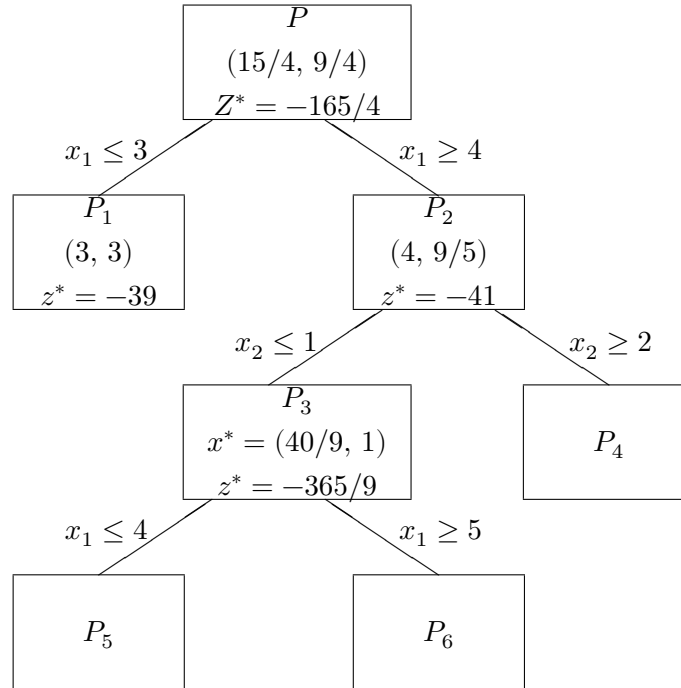
		-8	-5	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	
x_2	-5	0	1	0	0	0	1	1
x_1	-8	1	0	0	1/9	0	-5/9	40/9
x_3	0	0	0	1	-1/9	0	-4/9	5/9
x_5	0	0	0	0	1/9	1	-5/9	4/9
		0	0	0	-8/9	0	-5/9	-365/9

la solución óptima $(x^3)^* = (\frac{40}{9}, 1)$, con valor objetivo $z_3^* = -\frac{365}{9} = -40.55$.



Hemos perdido la integridad de la variable x_1 y como $-\frac{365}{9} < -39$, hemos de seguir ramificando. La política óptima del algoritmo es resolver las ramificaciones que se derivan de P_3 antes de sondear P_4 .

- Subproblema-5. (P_5). Se añade la restricción $x_1 \leq [x_1^*] \Rightarrow x_1 \leq 4$.
- Subproblema-6. (P_6). Se añade la restricción $x_1 \geq [x_1^*] + 1 \Rightarrow x_1 \geq 5$.



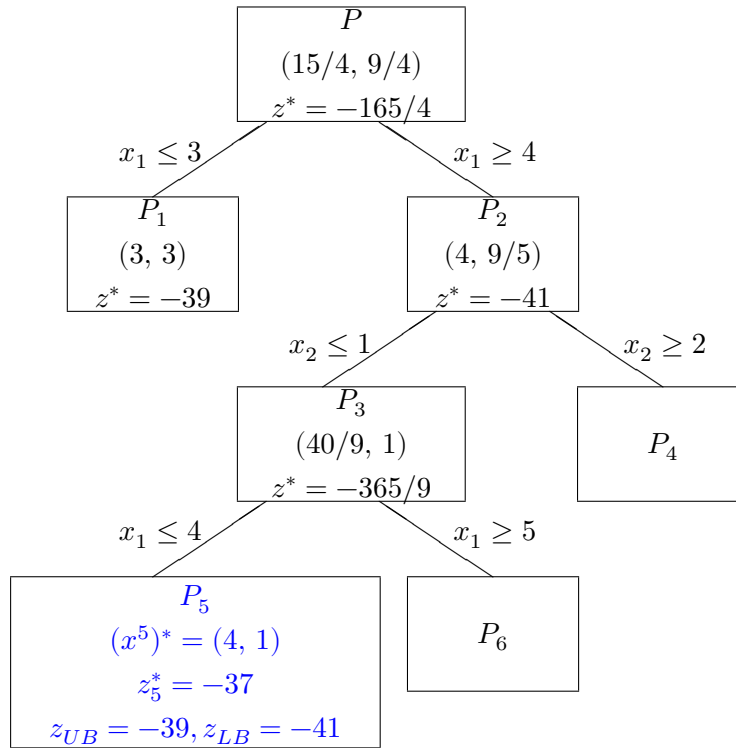
Veamos la resolución mediante el simplex de P_5 .

		-8	-5	0	0	0	0	0	0
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	0	0	1	0	1
x_1	-8	1	0	0	1/9	0	-5/9	0	40/9
x_3	0	0	0	1	-1/9	0	-4/9	0	5/9
x_5	0	0	0	0	1/9	1	-5/9	0	4/9
x_7	0	1	0	0	0	0	0	1	4
		0	0	0	-8/9	0	-5/9	0	-365/9

		-8	-5	0	0	0	0	0	0
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	0	0	1	0	1
x_1	-8	1	0	0	1/9	0	-5/9	0	40/9
x_3	0	0	0	1	-1/9	0	-4/9	0	5/9
x_5	0	0	0	0	1/9	1	-5/9	0	4/9
x_7	0	0	0	0	-1/9	0	5/9	1	-4/9
		0	0	0	-8/9	0	-5/9	0	-365/9

		-8	-5	0	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	0	0	1	0	1
x_1	-8	1	0	0	0	0	0	1	4
x_3	0	0	0	1	0	0	-1	-1	1
x_5	0	0	0	0	0	1	0	1	0
x_4	0	0	0	0	1	0	-5	-9	4
		0	0	0	0	0	-5	-8	-37

Resolviendo P_5 tenemos solución entera $(x^5)^* = (4, 1)$ con $z_5^* = -37$, por lo que es nodo terminal.-



Retomamos el problema P_6 , aplicando el algoritmo del simplex.

		-8	-5	0	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	0	0	1	0	1
x_1	-8	1	0	0	1/9	0	-5/9	0	40/9
x_3	0	0	0	1	-1/9	0	-4/9	0	5/9
x_5	0	0	0	0	1/9	1	-5/9	0	4/9
x_7	0	-1	0	0	0	0	0	1	-5
		0	0	0	-8/9	0	-5/9	0	-365/9

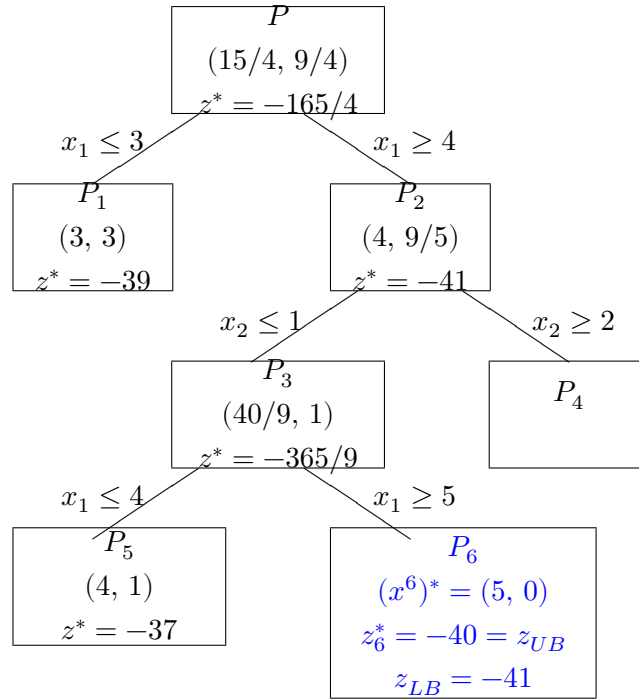
 $\Rightarrow$

		-8	-5	0	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	0	0	1	0	1
x_1	-8	1	0	0	1/9	0	-5/9	0	40/9
x_3	0	0	0	1	-1/9	0	-4/9	0	5/9
x_5	0	0	0	0	1/9	1	-5/9	0	4/9
x_7	0	0	0	0	1/9	0	-5/9	1	-5/9
		0	0	0	-8/9	0	-5/9	0	-365/9

 $\Rightarrow$

		-8	-5	0	0	0	0	0	
		x_1	x_2	x_3	x_4	x_5	x_6	x_7	
x_2	-5	0	1	0	1/5	0	0	9/5	0
x_1	-8	1	0	0	0	0	0	-1	5
x_3	0	0	0	1	-1/5	0	0	-4/5	1
x_5	0	0	0	0	0	1	0	-1	1
x_7	0	0	0	0	1/5	0	1	-9/5	1
		0	0	0	-1	0	0	-1	-40

Solución entera (5,0), por lo que es nodo terminal y además mejora la candidata a solución óptima que teníamos hasta ahora, pues $z^* = -40$.



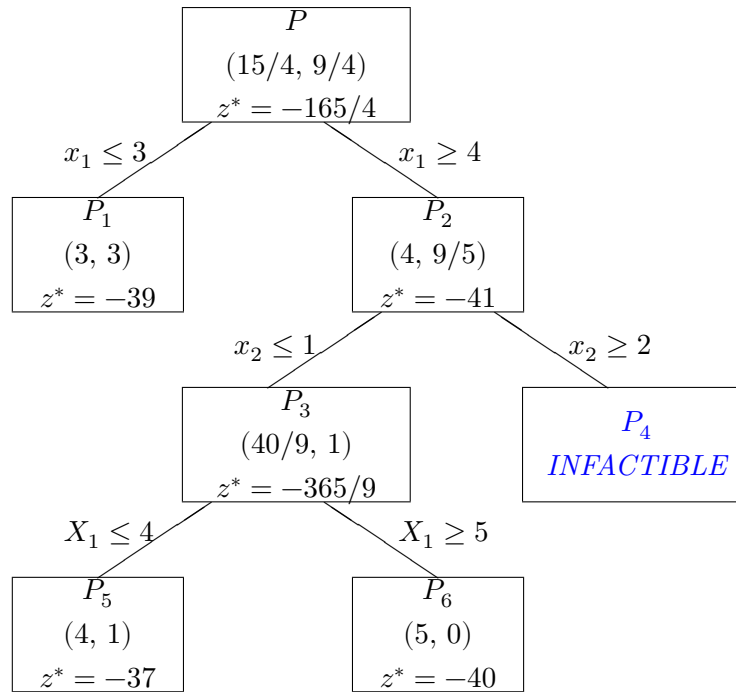
Ascendemos en el árbol y analizamos los nodos que habían quedado sin sondear. Como el óptimo asociado a P_2 , que no está totalmente sondeado, es $-41 < -40$, entonces no está acotado y tenemos que sondearlo.

Así pues resolvemos P_4 . La tabla del simplex queda, recordemos que hay que añadir la restricción a la tabla óptima de P_2 , como sigue:

		-8	-5	0	0	0	0	
		x_1	x_2	x_3	x_4	x_6	x_7	
x_2	-5	0	1	0	1/5	9/5	0	9/5
x_1	-8	1	0	0	0	-1	0	4
x_3	0	0	0	1	-1/5	-4/5	0	1/5
x_6	0	0	-1	0	0	0	1	-2
		0	0	0	-1	-1	0	-41

		-8	-5	0	0	0	0	
		x_1	x_2	x_3	x_4	x_6	x_6	
x_2	-5	0	1	0	1/5	9/5	0	9/5
x_1	-8	1	0	0	0	-1	0	4
x_3	0	0	0	1	-1/5	-4/5	0	1/5
x_6	0	0	0	0	1/5	9/5	1	-1/5
		0	0	0	-1	-1	0	-41

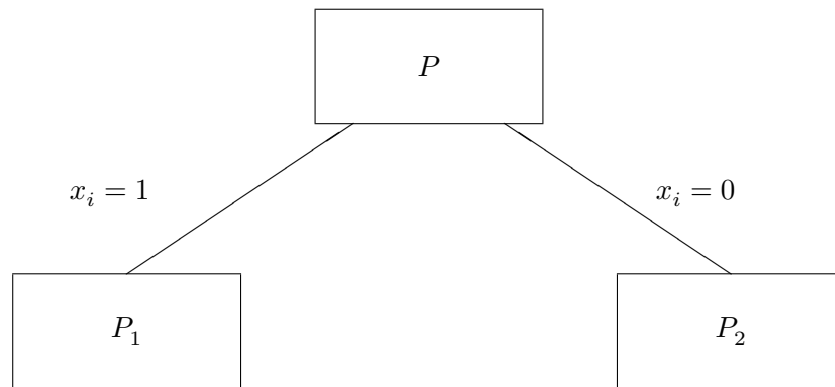
El problema es infactible.



Así pues el árbol queda completo. La solución óptima es, por tanto, $x^* = (x^6)^* = (5, 0)$, con $z^* = z_{UB} = -40$.

3.2.2 Programación Booleana

Como ya vimos, en ocasiones los problemas tienen restricciones en las que las variables sólo pueden tomar los valores $\{0,1\}$, que llamamos booleanos (p.e. el problema de asignación). Para resolverlo usaremos una variante del algoritmo de ramificación y acotación, adaptada a esa particularidad. La ramificación será en este caso el algoritmo de Balas.



Para facilitar la resolución del problema actuaremos como sigue:

1. Si es un problema de minimizar, lo transformamos en uno de maximizar, pues es más fácil comparar cantidades positivas.
2. Cambiar los costos de forma que todos sean positivos. Así, si $c_j < 0$, entonces hacemos el cambio de variable $y_j = 1 - x_j$.
3. Se efectúan las ramificaciones con el orden de las variables de mayor costo de las que no tengan valor asignado.
4. Se usa como valor de acotación el mayor de los posibles, suponiendo que todas las variables que no tienen valor asignado valen 1.

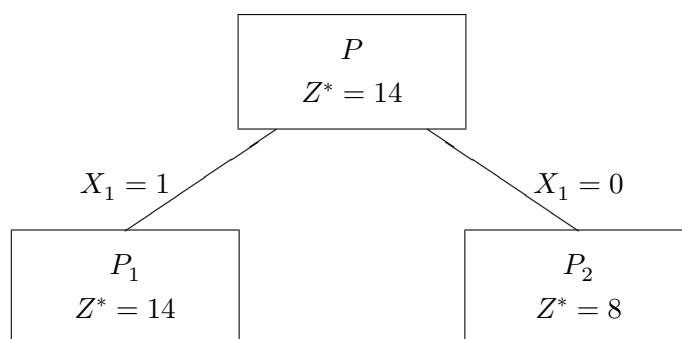
Transformamos el problema, de acuerdo a lo referido anteriormente.

$$\begin{array}{ll}
 \text{Max} & 6x_1 + 5x_2 - x_3 + 2x_4 \\
 \text{s.a:} & 3x_1 + 4x_2 + x_3 + x_4 \geq 4 \\
 & 2x_1 + x_2 - x_3 - x_4 \leq 3 \\
 & 4x_1 + 2x_2 - 2x_3 + 2x_4 \leq 5 \\
 & x_i \geq 0, \ i = 1, 2, 3, 4. \\
 & x_i \text{ booleanas}, \ i = 1, 2, 3, 4.
 \end{array}
 \quad \Rightarrow \quad
 \begin{array}{ll}
 \text{Max} & 6x_1 + 5x_2 + y_3 + 2x_4 - 1 \\
 \text{s.a:} & 3x_1 + 4x_2 - y_3 + x_4 \geq 3 \\
 & 2x_1 + x_2 + y_3 - x_4 \leq 4 \\
 & 4x_1 + 2x_2 + 2y_3 + 2x_4 \leq 7 \\
 & x_i \geq 0, \ i = 1, 2, 4, \ y_3 \geq 0. \\
 & x_i \text{ booleanas}, \ i = 1, 2, 4, \ y_3 \text{ booleana}.
 \end{array}$$

Iniciamos las ramificaciones.

Subproblema-1 (P_1) $\equiv x_1 = 1, z^* = 14$.

Subproblema-2 (P_2) $\equiv x_1 = 0, z^* = 8$.



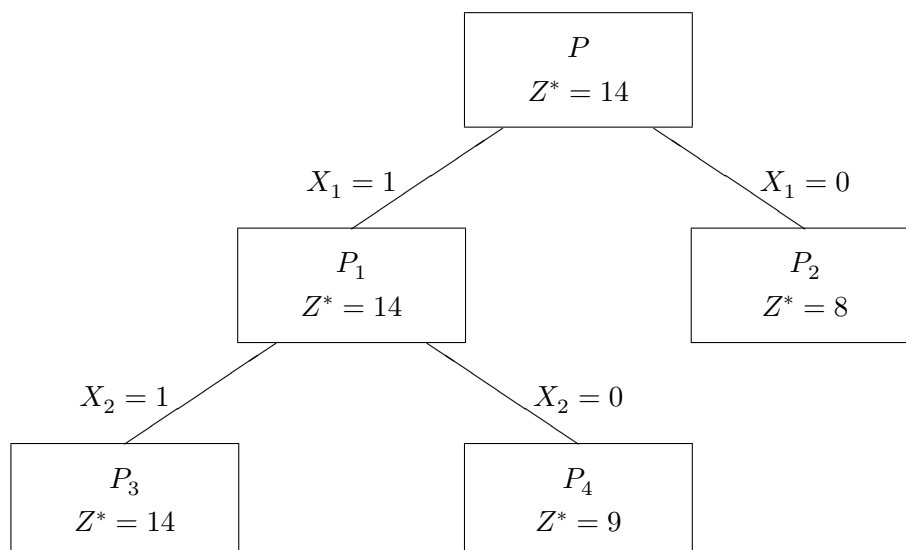
Comenzamos por P_1 , que es el más prometedor. Las ecuaciones quedan:

$$\begin{aligned}
 4x_2 - y_3 + x_4 &\geq 0 \\
 x_2 + y_3 - x_4 &\leq 2 \\
 2x_2 + 2y_3 + 2x_4 &\leq 3 \\
 x_i &\geq 0, \quad i = 2, 4, \quad y_3 \geq 0. \\
 x_i &\text{ booleanas, } i = 2, 4, \quad y_3 \text{ booleana.}
 \end{aligned}$$

Continuamos las ramificaciones, a partir de P_1 .

Subproblema-3 (P_3) $\equiv x_2 = 1, z^* = 14$.

Subproblema-4 (P_4) $\equiv x_2 = 0, z^* = 9$.



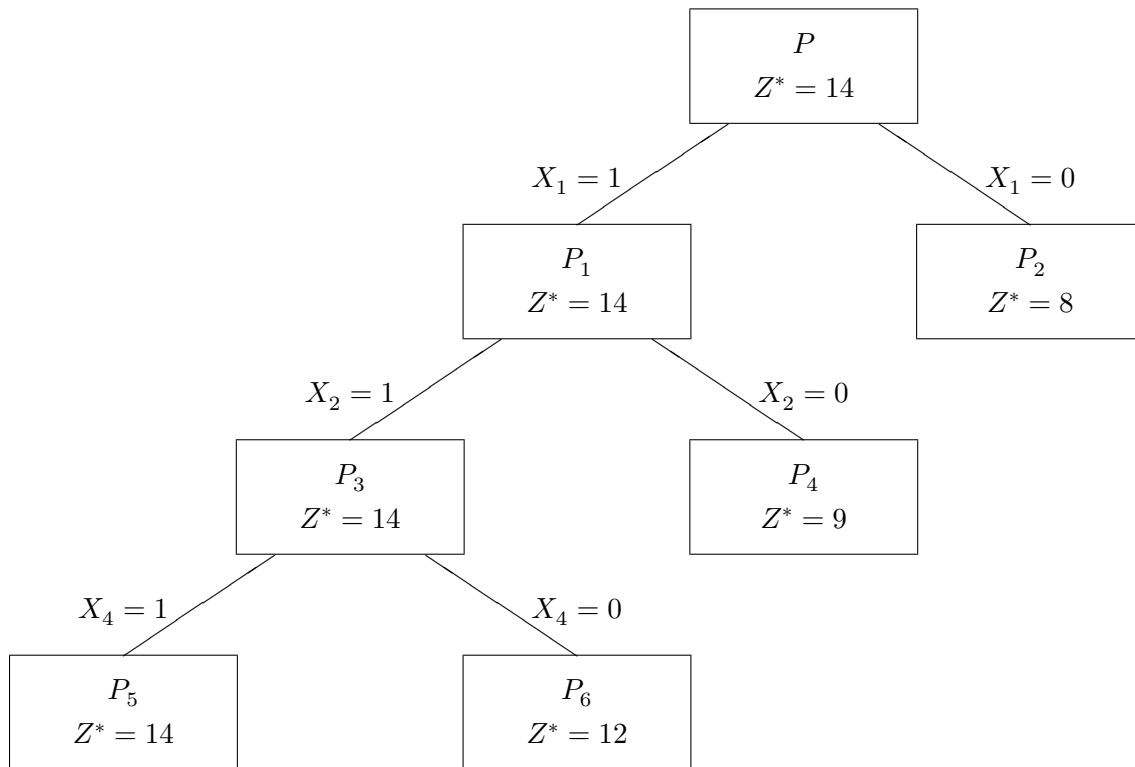
Seguimos con P_3 , que es el más prometedor. Las ecuaciones quedan:

$$\begin{aligned} -y_3 + x_4 &\geq -4 \\ y_3 - x_4 &\leq 1 \\ 2y_3 + 2x_4 &\leq 1 \\ x_4, y_3 &\geq 0 \\ x_4, y_3 &\text{ booleanas} \end{aligned}$$

Continuamos las ramificaciones por P_3 .

Subproblema-5 ($P_5 \equiv x_4 = 1, z^* = 14$).

Subproblema-6 ($P_6 \equiv x_4 = 0, z^* = 12$).

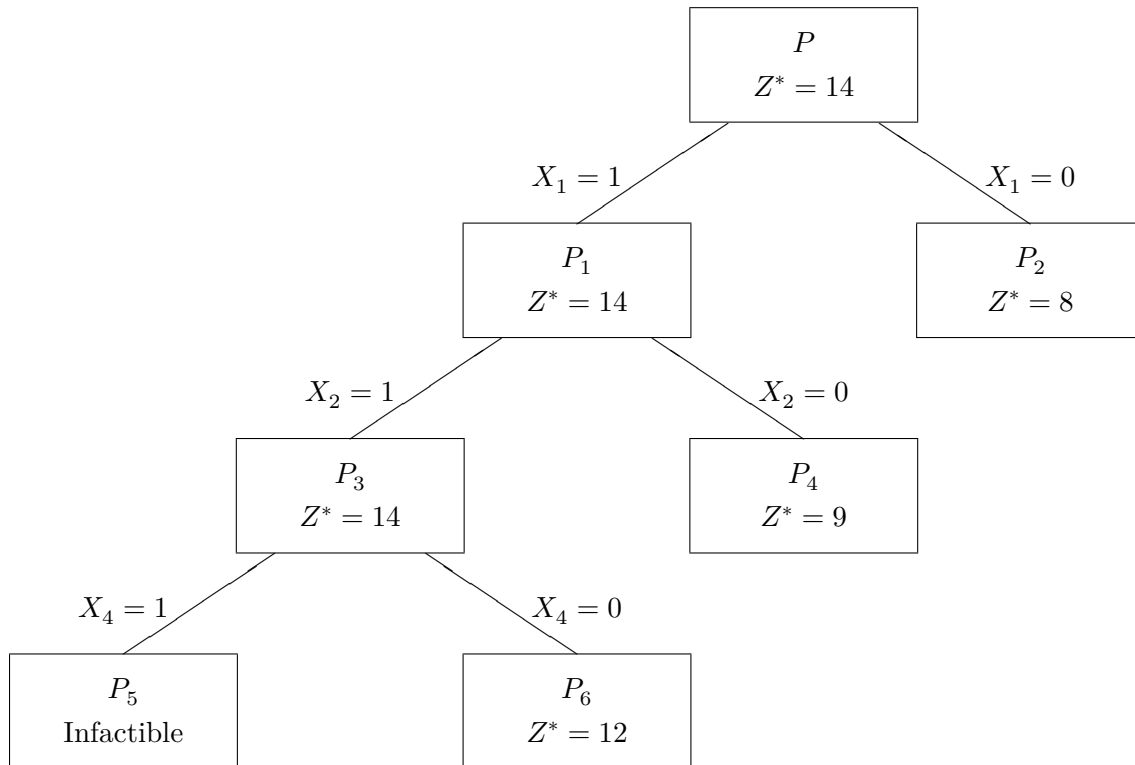


Seguimos con P_5 , que es el más prometedor. Las ecuaciones quedan:

$$\begin{aligned} -y_3 &\geq -5 \\ y_3 &\leq 2 \\ 2y_3 &\leq -1 \\ y_3 &\geq 0 \\ y_3 &\text{ booleana.} \end{aligned}$$

El problema es infactible, como podemos ver en la tercera ecuación.

Con lo que podemos completar el árbol.



Subimos un nivel y ramificamos desde P_6 , cuyas ecuaciones son:

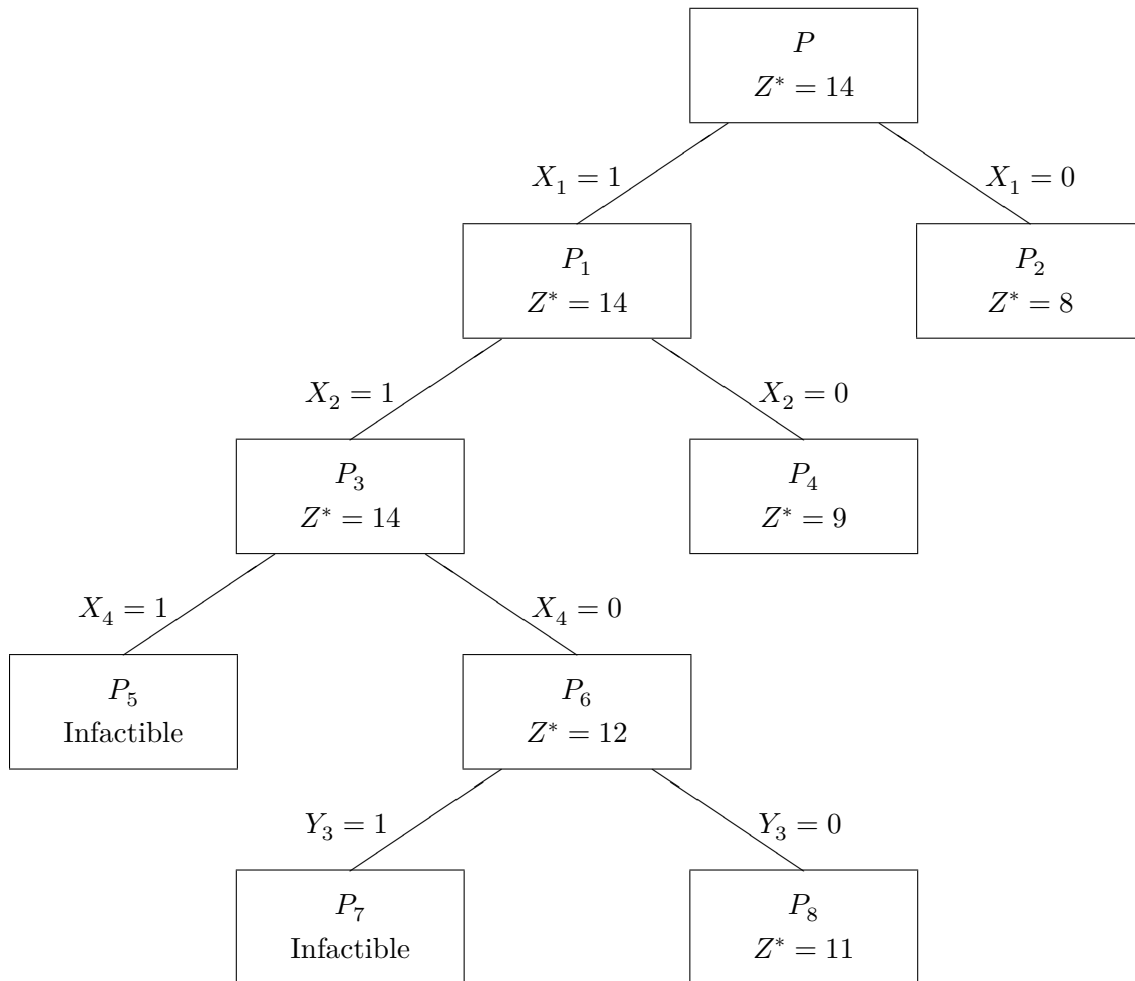
$$\begin{aligned}
 -y_3 &\geq -4 \\
 y_3 &\leq 1 \\
 2y_3 &\leq 1 \\
 y_3 &\geq 0 \\
 y_3 &\text{ booleana.}
 \end{aligned}$$

Subproblema-7 (P_7) $\equiv y_3 = 1, z^* = 12$.

Subproblema-8 (P_8) $\equiv y_3 = 0, z^* = 11$.

Obviamente P_7 es infactible por la tercera ecuación, por lo que tenemos una candidata a solución óptima en P_8 con $x_1 = 1, x_2 = 1, x_4 = 0, y_3 = 0$, que verifican todas las restricciones y valor de la función objetivo $z^*=11$.

El árbol queda ahora



Si analizamos los nodos que no hemos sondeado (P_2 y P_4) vemos que su función objetivo está acotada por la solución que ya tenemos, por lo que es peor. Así pues, no hay que sondearlas y esa es la óptima. Para acabar, debemos deshacer los cambios. Así tenemos que $(1,1,1,0)$ es la solución y el valor de la función objetivo original es $z^* = -11$.

3.3 Dual del Simplex y Simplex Columna

3.3.1 Dual del Simplex

3.3.1.1 Introducción

Este algoritmo se usa para evitar, en determinadas ocasiones, las variables artificiales y para el análisis de sensibilidad, que veremos posteriormente. Consideremos el siguiente problema.

Ejemplo 3.3.1.

$$\begin{array}{ll} \text{Min} & 2x_1 + 3x_2 + 4x_3 \\ \text{s.a:} & x_1 + 2x_2 + x_3 \geq 3 \\ & 2x_1 - x_2 + 3x_3 \geq 4 \\ & x_i \geq 0, \ i = 1, 2, 3. \end{array}$$

Cambiamos las inecuaciones de signo e introducimos variables de holgura.

$$\begin{array}{ll} \text{Min} & 2x_1 + 3x_2 + 4x_3 \\ \text{s.a:} & -x_1 - 2x_2 - x_3 + x_4 = -3 \\ & -2x_1 + x_2 - 3x_3 + x_5 = -4 \\ & x_i \geq 0, \ i = 1, 2, 3, 4, 5. \end{array}$$

Este planteamiento nos conduce a la siguiente tabla.

		2	3	4	0	0	
x_4	0	-1	-2	-1	1	0	-3
x_5	0	-2	1	-3	0	1	-4
		-2	-3	-4	0	0	0

En la tabla tenemos las condiciones de optimalidad ($z_j - c_j \leq 0$), pero no estamos en un punto factible ya que los b_i no son mayores o iguales a cero. En estas condiciones decimos que la base es **dual factible**. Vamos a ver un algoritmo que consigue la factibilidad sin perder la optimalidad que ya tenemos.

3.3.1.2 Algoritmo dual del Simplex

Consideremos el problema (PPL) de minimizar.

1. Encontrar una base B del primal dual factible ($z_j - c_j \leq 0$).
2. Si $\bar{b} \geq 0$, entonces STOP. Estamos en el óptimo. En otro caso, sea

$$\bar{b}_r = \min_i \{b_i / b_i < 0\}$$

3. Si $y_{rj} \geq 0$ para cada j, entonces el dual es no acotado $\Rightarrow$ primal es infactible. En otro caso, sea

$$\frac{z_k - c_k}{y_{rk}} = \min_j \left\{ \frac{z_j - c_j}{y_{rj}} \text{ tal que } y_{rj} < 0 \right\}$$

4. Pivotamos sobre y_{rk} y volvemos al paso 2

Observación 3.3.1. *La variable que sale de la base, sólo tiene que ser negativa, escoger el mínimo (la que peor cumple la condición de factibilidad) se hace con el fin de acelerar el algoritmo. Al igual que con el simplex, tenemos un cierto margen de discrecionalidad para elegir la variable que sale en la base, pero una vez determinada ésta, la variable que entra no admite dicha discrecionalidad, salvo caso de empate en la condición de entrada.*

Observación 3.3.2. *¿Cómo quedaría el algoritmo si el problema fuera de maximizar? Igual que en el caso anterior, salvo en el paso 3., que quedaría:*

Si $y_{rj} \geq 0$ para cada j, entonces el dual es no acotado $\Rightarrow$ primal es infactible. En otro caso, sea

$$\frac{z_k - c_k}{y_{rk}} = \max_j \left\{ \frac{z_j - c_j}{y_{rj}}, \text{ tal que } y_{rj} < 0 \right\},$$

pues hay que recordar que en ese caso el vector $z_j - c_j \geq 0$ y por tanto la fracción es negativa, pues el numerador es positivo y el denominador negativo, así se trata de elegir la fracción menos negativa y, por tanto, el máximo.

Retomemos el ejemplo 3.2.6 y apliquemos el algoritmo dual del simplex.

$$\begin{aligned} \text{Min} \quad & 2x_1 + 3x_2 + 4x_3 \\ \text{s.a:} \quad & -x_1 - 2x_2 - x_3 + x_4 = -3 \\ & -2x_1 + x_2 - 3x_3 + x_5 = -4 \\ & x_i \geq 0, \quad i = 1, 2, 3, 4, 5. \end{aligned}$$

Este planteamiento nos conduce, como hemos visto, a la siguiente tabla.

		2	3	4	0	0	
x_4	0	-1	-2	-1	1	0	-3
x_5	0	-2	1	-3	0	1	-4
		-2	-3	-4	0	0	0

Observación 3.3.3. Obsérvese que estamos en las condiciones de partida del algoritmo dual, pues tenemos optimalidad ($z_j - c_j \leq 0$) en un problema de minimizar, pero no tenemos la factibilidad, pues $\bar{b} \not\geq 0$.

Sale la base x_5 , que es el que peor cumple la condición de factibilidad (el más negativo). Miramos su fila en la matriz y encontramos dos valores negativos, así calculamos:

$$\text{Min} \left\{ \frac{-2}{-2}, \frac{-4}{-3} \right\} = \left\{ 1, \frac{4}{3} \right\} = 1$$

Entra x_1 .

		2	3	4	0	0	
x_4	0	0	-5/2	1/2	1	-1/2	-1
x_1	2	1	-1/2	3/2	0	-1/2	2
		0	-4	-1	0	-1	4

Observación 3.3.4. Obsérvese que no se pierde la optimalidad y eso significa que hemos pivotado adecuadamente.

Sale x_4 y calculamos:

$$\text{Min} \left\{ \frac{-4}{-5/2}, \frac{-1}{-1/2} \right\} = \left\{ \frac{8}{5}, 2 \right\} = 8/5$$

Entra x_2 .

		2	3	4	0	0	
x_2	3	0	1	-1/5	-2/5	1/5	2/5
x_1	2	1	0	7/5	-1/5	-2/5	11/5
		0	0	-9/5	-8/5	-1/5	28/5

Con lo que hemos recuperado la factibilidad sin perder la optimalidad y, por tanto, hemos resuelto el problema. Solución $(\frac{11}{5}, \frac{2}{5}, 0)$.

3.3.2 Simplex Columna

Como hemos observado en los temas anteriores las tablas del simplex arrastran mucha información, que en el caso de los problemas enteros las hace difícilmente manejables. Para tratar de evitar esta situación estudiaremos una nueva manera de presentar las tablas del simplex que llamaremos **simplex columna** que en ocasiones (especialmente si tenemos pocas variables y muchas restricciones) es más eficiente. Como ventaja adicional tenemos que al introducir una restricción sólo aumentamos el número de filas, pero no el de columnas. El simplex columna se puede aplicar como el simplex habitual o el dual del simplex. En caso de tener que introducir variables artificiales es mejor resolver el problema aplicando la forma tabular habitual y trasladar la tabla óptima a simplex columna con el fin de iniciar los cortes necesarios para resolver el problema entero. Consideremos un (PPL).

$$\begin{array}{ll} \text{Max} & c^t x \\ \text{s.a.} & Ax \leq b \\ & x \geq 0 \end{array} \implies \begin{array}{ll} \text{Max} & c^t x \\ \text{s.a.} & Ax + Ix_h = b \\ & x, x_h \geq 0 \end{array}$$

El simplex columna, en general, tiene la siguiente forma:

	x_{NB}	
x_{NB}	0	$-I$
x_B	$B^{-1}b$	$B^{-1}A$
	z	$z_j - c_j$

En el caso de que se trate de la tabla inicial, donde $B^{-1} = I$, la tabla queda;

	x_{NB}	
x_{NB}	0	$-I$
x_B	b	A
	z	$-c$

Nota 3.3.1. El orden de las filas puede ser alterado sin que eso afecte a la validez de la tabla.

Veamos unos cuantos ejemplos.

Ejemplo 3.3.2. *Simplex habitual*

$$\begin{array}{ll}
\text{Max} & 8x_1 + 5x_2 \\
\text{s.a:} & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_i \geq 0, \ i = 1, 2.
\end{array}
\Rightarrow
\begin{array}{ll}
\text{Max} & 8x_1 + 5x_2 \\
\text{s.a:} & x_1 + x_2 + x_3 = 6 \\
& 9x_1 + 5x_2 + x_4 = 45 \\
& x_i \geq 0, \ i = 1, 2, 3, 4.
\end{array}$$

Las variables no básicas (x_{NB}) son x_1 y x_2 y el resto básicas. Así la primera tabla queda:

	x_1	x_2	
x_1	0	-1	0
x_2	0	0	-1
x_3	6	1	1
x_4	45	9	5
	0	-8	-5

Ahora aplicamos el simplex para decidir qué variable entra en la base y cual sale. En este caso la que peor cumple la condición de optimalidad es x_1 , que es la que entra y sale $\text{Min}\{\frac{6}{1}, \frac{45}{9}\} = \frac{45}{9}$, por tanto sale x_4 . O sea, lo mismo que en el simplex habitual. Así pues pivotamos sobre el elemento en negrita y buscamos recuperar la matriz -I. Además pivotando sobre el mismo elemento conseguimos un cero como valor de $\bar{b}_4$, lo que es lógico porque pasa a ser una variable no básica, que valen cero. En los pivotajes se incluye la última fila, con lo que se calcula de forma automática el vector $z_j - c_j$ y el valor de la función objetivo. La tabla queda, por tanto:

	x_4	x_2			x_4	x_3		
x_1	5	1/9	5/9	$\Rightarrow$	x_1	15/4	1/4	-5/4
x_2	0	0	-1		x_2	9/4	-1/4	9/4
x_3	1	-1/9	4/9		x_3	0	0	-1
x_4	0	-1	0		x_4	0	-1	0
	40	8/9	-5/9			165/4	3/4	5/4

Estamos en el óptimo $(\frac{15}{4}, \frac{9}{4})$.

Nota 3.3.2. *Obsérvese que las tablas contienen la misma información que la forma tabular habitual.*

Ejemplo 3.3.3. *Algoritmo dual del simplex*

$$\begin{array}{ll}
\text{Min} & 2x_1 + 3x_2 + 4x_3 \\
\text{s.a:} & x_1 + 2x_2 + x_3 \geq 3 \\
& 2x_1 - x_2 + 3x_3 \geq 4 \\
& x_i \geq 0, \ i = 1, 2, 3.
\end{array}
\Rightarrow
\begin{array}{ll}
\text{Min} & 2x_1 + 3x_2 + 4x_3 \\
\text{s.a:} & -x_1 - 2x_2 - x_3 + x_4 = -3 \\
& -2x_1 + x_2 - 3x_3 + x_5 = -4 \\
& x_i \geq 0, \ i = 1, 2, 3, 4, 5.
\end{array}$$

La tabla queda:

	x_1	x_2	x_3		x_5	x_2	x_3		x_5	x_4	x_3			
x_1	0	-1	0	0	x_1	2	-1/2	-1/2	3/2	x_1	11/5	-2/5	-1/5	7/5
x_2	0	0	-1	0	x_2	0	0	-1	0	x_2	2/5	1/5	-2/5	-1/5
x_3	0	0	0	-1	x_3	0	0	0	-1	x_3	0	0	0	-1
x_4	-3	-1	-2	-1	x_4	-1	-1/2	-5/2	1/2	x_4	0	0	-1	0
x_5	-4	-2	1	-3	x_5	0	-1	0	0	x_5	0	-1	0	0
	0	-2	-3	-4		4	-1	-4	-1		28/5	-1/5	-8/5	-9/5

Estamos en el óptimo $(\frac{11}{5}, \frac{2}{5}, 0)$.

Ejemplo 3.3.4. *Problema con variables artificiales*

$$\begin{array}{ll}
\text{Min} & x_1 - 2x_2 \\
\text{s.a:} & x_1 + x_2 \leq 4 \\
& x_1 - x_2 \geq 1 \\
& x_i \geq 0, \ i = 1, 2.
\end{array}
\Rightarrow
\begin{array}{ll}
\text{Min} & x_1 - 2x_2 + My_1 \\
\text{s.a:} & x_1 + x_2 + x_3 = 4 \\
& x_1 - x_2 - x_4 + y_1 = 1 \\
& x_i \geq 0, \ i = 1, 2, 3, 4. \\
& y_1 \geq 0
\end{array}$$

Las tabla óptima es:

		1	-2				M
		x_1	x_2	x_3	x_4	y_1	
x_2	-2	0	1	1/2	1/2	-1/2	3/2
x_1	1	1	0	1/2	-1/2	1/2	5/2
		0	0	-1/2	-3/2	$-M - 1/2$	-1/2

Por lo que su tabla óptima del simplex columna queda:

		x_3	x_4
x_1	5/2	1/2	-1/2
x_2	3/2	1/2	1/2
x_3	0	-1	0
x_4	0	0	-1
	-1/2	-1/2	-3/2

3.4 Algoritmos de Corte

Los métodos de corte, buscan una reestructuración del espacio de la solución factible imponiendo algunas restricciones (especialmente diseñadas) en el espacio original de modo que el punto factible óptimo requerido se exprese como un punto extremo adecuado del espacio de solución modificado. La idea general es que estas restricciones adicionales corten sistemáticamente porciones del espacio de solución tal que nunca se excluyan puntos factibles.

Los métodos de corte son en realidad las primeras técnicas sistemáticas desarrolladas para el problema de números enteros (lineales). Los primeros trabajos de Dantzig et al (1954) y Markowitz y Manne (1957) dirigieron la atención de los investigadores a la importancia de resolver programas lineales en números enteros, pero Dantzig (1959) fue el primero en proponer un corte para resolver tales problemas. Su idea era resolver primero el programa lineal ignorando las condiciones de los enteros. Si la solución básica resultante no es entera, entonces se debe asegurar un nuevo conjunto de valores para las variables no básicas. Esto se logra al darse cuenta de que la suma de las variables enteras no básicas debe ser al menos igual a uno; es decir, si x_j , $j \in \mathbb{NBI}$ son las variables enteras no básicas, entonces

$$\sum_{j \in \mathbb{NBI}} x_j \geq 1, \quad x_j \geq 0, \\ j \in \mathbb{NBI}$$

debe quedar satisfecho.

El primer algoritmo de corte finito fue desarrollado por Gomory (1958) para el problema de los enteros puros.

Hiperplanos de corte

Como en el caso de los algoritmos de ramificación y acotación se comienza por resolver el problema relajado. Para las variables que han de ser enteras se introduce una nueva restricción (corte) que representa una condición necesaria de integridad para esa variable. Esa restricción suprime una parte de la región factible del problema relajado (incluyendo la solución óptima), pero sin eliminar soluciones factibles (enteras) del problema original. Resumiendo, tenemos que el hiperplano de corte:

1. Deja fuera la solución óptima del problema relajado, con lo que hay que aplicar el dual del simplex.
2. Es verificado por todas las soluciones factibles del problema original.

3.4.1 Corte todo entero de Gomory (f-corte)

Esta técnica sólo se puede aplicar a problemas enteros puros con coeficientes de las restricciones enteros. Esto se debe a que el algoritmo no distingue entre variables regulares y las de holgura, en el sentido de que todas deben ser enteras. Luego si existen coeficientes no enteros en las restricciones podría haber soluciones enteras (variables regulares enteras y de holgura fraccionarias) que fueran eliminadas. Vamos a deducir teóricamente el corte fraccional o f-corte de Gomory. Sea el problema entero puro y el problema relajado en forma estándar

$$\text{Minimizar } c^T x$$

$$\text{s.a: } Ax = b$$

$$x \geq 0$$

$$x \in \mathbb{Z}^n$$

$$\text{Maximizar } c^T x$$

$$\text{s.a: } Ax = b$$

$$x \geq 0$$

$$x \in \mathbb{R}^n$$

Sea $x^* = (x_B^*, x_N^*)$ la solución óptima del problema relajado, por tanto solución básica factible, con $x_B^* = B^{-1}b$ y $x_N^* = 0$, donde $B = (a_1 | \dots | a_m)$ es la base óptima asociada. Supongamos que la componente i -ésima de $x_B^* = B^{-1}b = \bar{b}$ a la que notamos por $\bar{b}_i$ no toma un valor entero. Consideremos x un punto factible cualquiera, y reordenamos el vector $x = (x_B, x_N)$ de la misma manera que x^* , aunque ahora no podemos afirmar que $x_N = 0$.

Por ser x factible tenemos que $Ax = b \Rightarrow$

$$\begin{aligned} (B, N) \begin{pmatrix} x_B \\ x_N \end{pmatrix} &= b \Rightarrow Bx_B + Nx_N = b \\ &\Rightarrow Bx_B = b - Nx_N \\ &\Rightarrow x_B = B^{-1}b - B^{-1}Nx_N = \bar{b} - B^{-1}Nx_N \end{aligned}$$

Por lo que en la componente básica i se verifica que:

$$x_i = \bar{b}_i - \sum_{j \in NB} y_{ij}x_j \quad (1), \quad \text{donde } y_j = B^{-1}a_j \text{ es un vector de } m \text{ componentes.}$$

Sea y_{ij} la componente i del vector y_j .

$$\begin{aligned} \bar{b}_i &= [\bar{b}] + f_i, \quad 1 > f_i > 0, \\ y_{ij} &= [y_{ij}] + f_{ij}, \quad 1 > f_{ij} \geq 0 \quad (2). \end{aligned}$$

Los y_{ij} no pueden ser todos enteros ya que si lo fueran el problema sería infactible. Por (1), para toda solución factible

$$\left. \begin{aligned} x_i &= \bar{b}_i - \sum_{j \in NB} y_{ij}x_j \\ \bar{b}_i &\notin \mathbb{Z} \\ x_j \in \mathbb{Z} &\Rightarrow \sum_{j \in NB} y_{ij}x_j \in \mathbb{Z} \end{aligned} \right\} \Rightarrow x_i \notin \mathbb{Z} \Rightarrow \text{problema entero infactible.}$$

De (1) y (2), para todo x factible, la componente i -ésima,

$$\begin{aligned} x_i &= [\bar{b}_i] + f_i - \sum_{j \in NB} ([y_{ij}] + f_{ij})x_j, \\ x_i - \bar{b}_i - \sum_{j \in NB} [y_{ij}]x_j &= f_i - \sum_{j \in NB} f_{ij}x_j. \end{aligned}$$

Para que $x_i \in \mathbb{Z} \Rightarrow f_i - \sum_{j \in NB} f_{ij}x_j \in \mathbb{Z}$.

Como $x_j \geq 0$ y $f_{ij} \geq 0$, se tiene que $\sum_{j \in NB} f_{ij}x_j \geq 0$. Luego

$$f_i - \sum_{j \in NB} f_{ij}x_j \leq f_i < 1$$

Para que

$$1 > f_i - \sum_{j \in NB} f_{ij} x_j \in \mathbb{Z}$$

, debe ocurrir que $f_i - \sum_{j \in NB} f_{ij} x_j \leq 0$, luego para que x_i sea entera es condición necesaria que

$$f_i \leq \sum_{j \in NB} f_{ij} x_j$$

que cambiando de signo e introduciendo una variable de holgura (h_i , entera no negativa) la restricción a introducir es:

$$h_i - \sum_{j \in NB} f_{ij} x_j = -f_i \quad : = \text{corte fraccional (f-corte) de Gomory}$$

Con esta nueva restricción la solución actual no es factible, por lo que resolvemos por el dual del simplex. Si obtenemos solución entera, paramos, si no, seguimos introduciendo cortes. Si en alguna iteración se obtiene dual ilimitado, entonces, no existe solución factible entera.

Ejemplo 3.4.1. *Retomamos el problema*

$$\begin{array}{ll} \text{Max} & 8x_1 + 5x_2 \\ \text{s.a:} & x_1 + x_2 \leq 6 \\ & 9x_1 + 5x_2 \leq 45 \\ & x_i \geq 0, \quad i = 1, 2. \\ & x_i \text{ enteras} \quad i = 1, 2. \end{array} \quad \Rightarrow \quad \begin{array}{ll} \text{Max} & 8x_1 + 5x_2 \\ \text{s.a:} & x_1 + x_2 + x_3 = 6 \\ & 9x_1 + 5x_2 + x_4 = 45 \\ & x_i \geq 0, \quad i = 1, 2, 3, 4. \\ & x_i \text{ enteras} \quad i = 1, 2, 3, 4. \end{array}$$

Empezamos por resolver el problema relajado, cuya tabla óptima, como ya hemos visto, es

		x_4	x_3
x_1	15/4	1/4	-5/4
x_2	9/4	-1/4	9/4
x_3	0	0	-1
x_4	0	-1	0
	165/4	3/4	5/4

El corte lo hemos de dar sobre x_1 , ya que $f_1 = 3/4$ y $f_2 = 1/4$. Tenemos, por tanto,

$$\begin{aligned} f_{14} &= 1/4 - [1/4] = 1/4 - 0 = 1/4 \\ f_{13} &= -5/4 - [-5/4] = -5/4 + 2 = 3/4 \end{aligned}$$

Así pues la restricción queda:

$$h_1 - \frac{1}{4}x_4 - \frac{3}{4}x_3 = -\frac{3}{4}$$

Veamos como queda la restricción expresada en función de x_1 y x_2 , con el único finde ver la representación gráfica del corte. Sabemos que

$$\begin{aligned}x_3 &= 6 - x_1 - x_2 \\x_4 &= 45 - 9x_1 - 5x_2\end{aligned}$$

Por lo que sustituyendo en el corte nos queda:

$$h_1 - \frac{1}{4}(45 - 9x_1 - 5x_2) - \frac{3}{4}(6 - x_1 - x_2) = -\frac{3}{4}$$

$$h_1 - \frac{45}{4} + \frac{9}{4}x_1 + \frac{5}{4}x_2 - \frac{18}{4} + \frac{3}{4}x_1 + \frac{3}{4}x_2 = -\frac{3}{4}$$

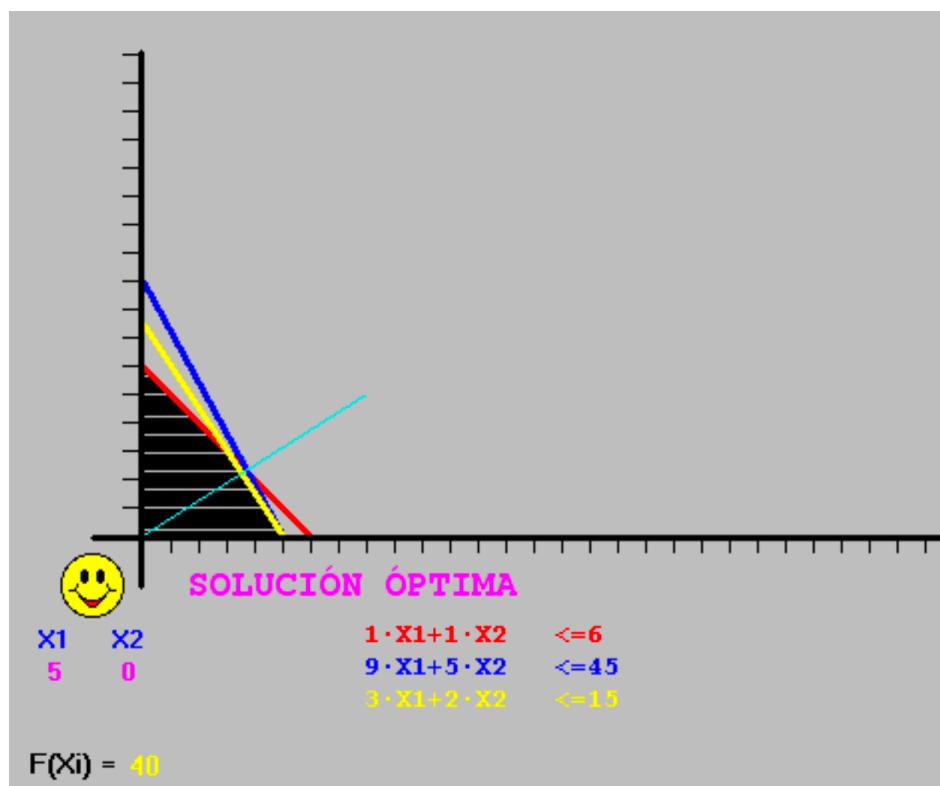
Lo pasamos a restricción:

$$-\frac{45}{4} + \frac{9}{4}x_1 + \frac{5}{4}x_2 - \frac{18}{4} + \frac{3}{4}x_1 + \frac{3}{4}x_2 \leq -\frac{3}{4}$$

Y finalmente nos queda:

$$3x_1 + 2x_2 \leq 15$$

Con lo que gráficamente nos queda:



Retomemos la tabla óptima e introducimos el corte calculado:

		x_4	x_3			x_4	h_1	
x_1	15/4	1/4	-5/4	$\Rightarrow$	x_1	5	2/3	-5/3
x_2	9/4	-1/4	9/4		x_2	0	-1	3
x_3	0	0	-1		x_3	1	1/3	-4/3
x_4	0	-1	0		x_4	0	-1	0
h_1	-3/4	-1/4	-3/4		h_1	0	0	-1
	165/4	3/4	5/4			40	1/3	5/3

Tenemos la solución óptima (5,0).

3.4.2 Corte mixto de Gomory(m-corte)

Esta técnica se aplica a problemas enteros mixtos, pues permite dar un corte a una variable que debe ser entera sin obligar a las demás a serlo. Sea x^* la solución óptima del problema relajado y x_i^* una variable básica no entera y que debería serlo. Para cualquier x factible, como vimos anteriormente, se verifica:

$$x_i = \lceil \bar{b}_i \rceil + f_i - \sum_{j \in NB} y_{ij} x_j \quad (3)$$

Sean $J^+ = \{j \in NB / y_{ij} \geq 0\}$ y $J^- = \{j \in NB / y_{ij} < 0\}$

Para que x_i sea entera: $x_i \leq \lceil \bar{b}_i \rceil$ o $x_i \geq \lceil \bar{b}_i \rceil + 1$, pero no se pueden dar las dos posibilidades.

- En el caso de que $x_i \leq \lceil \bar{b}_i \rceil$, de la ecuación (3):

$$\lceil \bar{b}_i \rceil + f_i - \sum_{j \in NB} y_{ij} x_j \leq \lceil \bar{b}_i \rceil \Rightarrow f_i \leq \sum_{j \in NB} y_{ij} x_j \Rightarrow f_i \leq \sum_{j \in J^+} y_{ij} x_j - \sum_{j \in J^-} y_{ij} x_j$$

Como $\sum_{j \in J^-} y_{ij} x_j \leq 0$, porque $x_j \geq 0$ por ser factibles, entonces:

$$f_i \leq \sum_{j \in J^+} y_{ij} x_j \Rightarrow -f_i \geq - \sum_{j \in J^+} y_{ij} x_j$$

- En el caso de que $x_i \geq \lceil \bar{b}_i \rceil + 1$, de la ecuación (3):

$$\lceil \bar{b}_i \rceil + f_i - \sum_{j \in NB} y_{ij} x_j \geq \lceil \bar{b}_i \rceil + 1 \Rightarrow f_i - 1 \geq \sum_{j \in NB} y_{ij} x_j$$

$$\Rightarrow f_i - 1 \geq \sum_{j \in J^+} y_{ij} x_j + \sum_{j \in J^-} y_{ij} x_j$$

Como $\sum_{j \in J^+} y_{ij} x_j \geq 0$, porque $x_j \geq 0$ por ser factibles, tenemos:

$$f_i - 1 \geq \sum_{j \in J^-} y_{ij} x_j \Rightarrow \frac{f_i - 1}{1 - f_i} \geq \frac{1}{1 - f_i} \sum_{j \in J^-} y_{ij} x_j$$

ya que $1 - f_i > 0$. Entonces:

$$-1 \geq \frac{1}{1 - f_i} \sum_{j \in J^+} y_{ij} x_j \Rightarrow -f_i \geq \frac{f_i}{1 - f_i} \sum_{j \in J^+} y_{ij} x_j,$$

ya que $f_i > 0$.

Como las desigualdades anteriores no se pueden dar simultáneamente podemos incluirlas en la misma restricción, es decir, $x_i \notin [\lceil \bar{b}_i \rceil, \lceil \bar{b}_i \rceil + 1]$. Equivalentemente, $x_i \in [\bar{b}_i, \bar{b}_i + 1]$ lo que

implica

$$-\sum_{j \in J^+} y_{ij}x_j - \frac{f_i}{f_i - 1} \sum_{j \in J^-} y_{ij}x_j > -2f_i$$

Luego si $x_i \notin [\lceil \bar{b}_i \rceil, \lceil \bar{b}_i \rceil + 1]$ si

$$-\sum_{j \in J^+} y_{ij}x_j - \frac{f_i}{f_i - 1} \sum_{j \in J^-} y_{ij}x_j \leq -2f_i \leq -f_i$$

$$h_i - \sum_{j \in J^+} y_{ij}x_j - \frac{f_i}{f_i - 1} \sum_{j \in J^-} y_{ij}x_j = -f_i, \quad h_i \geq 0$$

La restricción anterior representa una condición necesaria para que x_i sea entera. Como en la tabla actual $x_j^* = 0$, $j \in NB$, la solución actual es infactible. Se aplicará por tanto el dual del simplex para resolverlo.

Ejemplo 3.4.2. Sea la tabla óptima de un problema relajado. En el problema original x_3 y x_4 tienen que ser enteras.

	x_1	x_2	x_3	x_4	
x_1	1	$-5/3$	0	$-1/3$	$5/3$
x_3	0	$-4/3$	1	$11/3$	$7/3$
	0	-5	0	-10	0

Lo pasamos a simplex columna:

	x_2	x_4
x_1	$5/3$	$-5/3$
x_2	0	-1
x_3	$7/3$	$-4/3$
x_4	0	-1
	0	-5

El m-corte de Gomory para x_3 sería:

$$\left. \begin{array}{l} J^+ = \{4\} \\ J^- = \{2\} \end{array} \right\} \Rightarrow h_3 - \frac{11}{3}x_4 - \frac{\frac{1}{3}}{\frac{1}{3} - 1} \left(-\frac{4}{3} \right) x_2 = -\frac{1}{3} \Rightarrow h_3 - \frac{11}{3}x_4 - \frac{2}{3}x_2 = -\frac{1}{3}$$

Lo metemos en la tabla:

		x_2	x_4
x_1	5/3	-5/3	-1/3
x_2	0	-1	0
x_3	7/3	-4/3	11/3
x_4	0	0	-1
h_3	-1/3	-2/3	-11/3
	0	-5	-10

Entraría h_3 y saldría x_4

		x_2	h_3
x_1	56/33	-53/33	-1/11
x_2	0	-1	0
x_3	2	-2	1
x_4	1/11	2/11	-3/11
h_3	0	0	-1
	10/11	-35/11	-30/11

x_4 debe ser entera y no lo es, por lo que introducimos el siguiente corte y lo añadimos a la tabla:

$$h_4 - \frac{2}{11}x_2 - \frac{\frac{1}{11}}{\frac{1}{11} - 1} \left(-\frac{3}{11} \right) h_3 = -\frac{1}{11} \Rightarrow h_4 - \frac{2}{11}x_2 - \frac{3}{110}h_3 = -\frac{1}{11}$$

La tabla quedaría:

		x_2	h_3
x_1	56/33	-53/33	-1/11
x_2	0	-1	0
x_3	2	-2	1
x_4	1/11	2/11	-3/11
h_3	0	0	-1
h_4	-1/11	-2/11	-3/110
	10/11	-35/11	-30/11

Entraría h_4 y saldría x_2

		h_4	h_3
x_1	5/2	-53/6	3/20
x_2	1/2	-11/2	3/20
x_3	1	-11	13/10
x_4	0	1	-3/10
h_3	0	0	-1
h_4	0	-1	0
	5/2	-35/2	-9/4

Tenemos la solución óptima (5/2,1/2).

3.4.3 Corte mixto de Gomory (m^* - corte)

El corte anterior no tiene en cuenta el hecho de que algunas x_j no básicas pueden ser enteras. Si esto se tiene en cuenta surge el siguiente corte más profundo (m^* - corte).

$$h_i = -f_i + \sum_{j \in NB} \lambda_j x_j \quad \text{con} \quad \lambda_j = \begin{cases} y_{ij} & \text{si } y_{ij} \geq 0 \text{ y } x_j \text{ no es entera,} \\ \frac{f_i}{f_i - 1} y_{ij} & \text{si } y_{ij} < 0 \text{ y } x_j \text{ no es entera,} \\ f_{ij} & \text{si } f_{ij} \leq f_i \text{ y } x_j \text{ es entera,} \\ \frac{f_i}{f_i - 1} (f_{ij} - 1) & \text{si } f_{ij} > f_i \text{ y } x_j \text{ es entera.} \end{cases}$$

Siguiendo el ejemplo del m-corte, el m^* corte sería:

$$h_3 - \frac{\frac{1}{3}}{\frac{1}{3} - 1} \left(-\frac{4}{3} \right) x_2 - \frac{\frac{1}{3}}{\frac{1}{3} - 1} \left(\frac{2}{3} - 1 \right) x_4 = -\frac{1}{3}$$

Metemos el corte en la tabla óptima:

		x_2	x_4
x_1	5/3	-5/3	-1/3
x_2	0	-1	0
x_3	7/3	-4/3	11/3
x_4	0	0	-1
h_3	-1/3	-2/3	-1/6
	0	-5	-10

Entra h_3 y sale x_2

		h_3	x_4
x_1	5/2	-5/2	1/12
x_2	1/2	-3/2	1/4
x_3	3	-1	4
x_4	0	0	-1
h_3	0	-1	0
	5/2	-15/2	-35/4

Tenemos la solución óptima $(5/2, 1/2)$.

Capítulo 4

Complejidad algorítmica

4.1 Introducción

La teoría de la complejidad permite clasificar los problemas según la dificultad con la que puedan resolverse mediante el uso de algoritmos.

Definición 4.1.1. *Un algoritmo es una representación finita de un método computacional para resolver un problema. Tiene la forma de un texto de extensión finita escrito en un lenguaje ad hoc, donde cada "frase" tiene un significado inequívoco. Este texto describe cálculos que se pueden realizar en un tiempo finito, utilizando un procedimiento de cálculo mecánico con memoria, sobre unos datos: la entrada o instancias del problema, debidamente codificados. Una entrada codificada de este tipo es una secuencia finita de símbolos de un alfabeto determinado (es decir, una palabra).*

En 1964, Cobham introdujo la clase $\mathcal{P}$ de problemas que son solucionables en tiempo polinomial, es decir, problemas solucionables mediante un algoritmo A satisfactorio: para cualquier n , $C(A, n) < p(n)$, donde p es un polinomio dado (entonces se dice que A corre en tiempo polinómico) y n es el tamaño del problema, y en los problemas lineales n es el número de variables. Ésta es la clase de problemas "fáciles". Hay problemas difíciles (también llamados "exponenciales") y, por supuesto, problemas que no se pueden resolver con un algoritmo (incorrectamente llamados indecidibles). Esta simplicidad no duró mucho.

En 1965, junto con la clase $\mathcal{P}$, Edmonds introdujo la clase $\mathcal{NP}$ y fue el primero en conjeturar que $\mathcal{P} \neq \mathcal{NP}$. Intuitivamente, esta clase $\mathcal{NP}$ corresponde a aquellos problemas que admiten una "prueba" algorítmica de longitud razonable y cuya validez puede verificarse en tiempo

polinomial. La sutileza aquí es que, a diferencia del caso de la clase $\mathcal{P}$, se supone que la prueba está dada. Por ejemplo, si consideramos un número entero m y si tomamos un número entero g , se puede verificar en tiempo polinomial si g divide o no a m (g es una “prueba” de la no primalidad de m). Sin embargo, no se conoce ningún algoritmo polinomial en $\log(m)$, de modo que se podría obtener la no primalidad de m . La conjetura de Edmonds ha sido considerada de manera formal por Cook [1971], y estudiada más a fondo por Karp [1972], la idea de Cook era que en la clase $\mathcal{NP}$ existen problemas que son “más difíciles” que otros, los problemas $\mathcal{NP}$ completos. Sería suficiente si solo hubiera un problema $\mathcal{NP}$ completo en $\mathcal{P}$ para concluir que $\mathcal{P} = \mathcal{NP}$. La conjetura de Edmonds fue la primera de muchas sobre toda una jerarquía de clases de complejidad.

Para los siguientes problemas se puede demostrar que tienen la Propiedad de Optimización Eficiente:

- El Problema de Dimensionamiento de Lotes sin Capacidad
- El Problema del Árbol de Peso Máximo
- El Problema de Emparejamiento de Peso Máximo
- El Problema del Camino Más Corto
- El Problema de Flujo Máximo
- El Problema de Programación Entera TU
- El Problema de Asignación

A continuación, hacemos esto más preciso: existe un algoritmo polinómico para estos problemas de optimización.

Por otro lado, hasta la fecha, nadie ha encontrado un algoritmo eficiente (polinómico) para ninguno de los siguientes problemas de optimización:

- El Problema de la Mochila 0-1
- El Problema de Cobertura de Conjuntos
- El Problema del Viajante de Comercio
- El Problema de Localización de Instalaciones sin Capacidad

- El Problema de Programación Entera
- El Problema del Árbol de Steiner
- El Problema de Corte de Stock

Este capítulo estará en gran parte dedicado a examinar cómo abordar problemas en este segundo grupo. Sin embargo, primero es útil discutir la distinción (real o imaginaria) entre estos dos grupos, para que cuando encontremos un nuevo problema de optimización tengamos una idea de cómo podríamos clasificarlo y luego intentar resolverlo.

Para desarrollar un método de clasificación, necesitamos solo cuatro conceptos:

Una clase C de problemas legítimos a los cuales se aplica la teoría.

Una subclase no vacía $C_A \subseteq C$ de problemas “fáciles”.

Una subclase no vacía $C_B \subseteq C$ de problemas “difíciles”.

Una relación “no más difícil que” entre pares de problemas legítimos.

Esto nos lleva inmediatamente a:

Proposición 4.1.1. (*Lema de Reducción*) *Supongamos que P y Q son dos problemas legítimos.*

Si Q es “fácil” y P “no es más difícil que” Q , entonces P es “fácil”.

Si P es “difícil” y P “no es más difícil que” Q , entonces Q es “difícil”.

4.2 Problemas de Decisión y las Clases $\mathcal{NP}$ y $\mathcal{P}$

Para definir la clase de problemas legítimos, es apropiado plantear problemas de decisión que tengan respuestas SÍ-NO. Así, un problema de optimización:

$$\max\{cx : x \in S\}$$

para el cual una instancia consiste en $\{c$ y una representación “estándar” de $S\}$ es reemplazado por el problema de decisión:

$$¿\text{Existe un } x \in S \text{ con valor } cx \geq k?$$

para el cual una instancia consiste en $\{c$, una representación “estándar” de S y un número

entero k }.

A continuación, damos una definición ligeramente más formal del tiempo de ejecución de un algoritmo. No solo el número de variables y restricciones o nodos y aristas define la longitud de la entrada, sino también el tamaño de los números que aparecen en los datos.

Definición 4.2.1. *Para una instancia de problema X , la longitud de la entrada $L = L(X)$ es la longitud de la representación binaria de una representación “estándar” de la instancia.*

Definición 4.2.2. *Dado un problema P , un algoritmo A para el problema, y una instancia X , sea $f_A(X)$ el número de cálculos elementales requeridos para ejecutar el algoritmo A en la instancia X . $f_A^*(l) = \sup_X \{f_A(X) : L(X) = l\}$ es el tiempo de ejecución del algoritmo A . Un algoritmo A es polinómico para un problema P si $f_A^*(l) = O(l^p)$ para algún entero positivo p .*

Ahora, podemos definir la clase de problemas “legítimos”.

Definición 4.2.3. $\mathcal{NP}$ es la clase de problemas de decisión con la propiedad de que para cualquier instancia en la que la respuesta es SÍ, existe una prueba “corta” (polinómica) del SÍ.

Notamos inmediatamente que si un problema de decisión asociado a un problema de optimización está en $\mathcal{NP}$, entonces el problema de optimización puede resolverse respondiendo el problema de decisión un número polinómico de veces (usando bisección en el valor de la función objetivo).

Proposición 4.2.1. *Para cada problema de optimización en las dos listas de la Sección 4.1, planteado como una maximización, el problema de decisión asociado es “¿Existe una solución primal con valor tan bueno o mejor que k ?” está en $\mathcal{NP}$.*

Ahora, podemos definir la clase de problemas “fáciles”.

Definición 4.2.4. $\mathcal{P}$ es la clase de problemas de decisión en $\mathcal{NP}$ para los cuales existe un algoritmo polinómico.

¿Los problemas de optimización en el segundo conjunto enumerado en la Sección 4.1 tienen algo en común aparte del hecho de que sus problemas de decisión están en $\mathcal{NP}$ y que nadie ha descubierto aún un algoritmo polinómico para ninguno de ellos?

Sorprendentemente, tienen una segunda propiedad en común: sus problemas de decisión están entre los problemas más difíciles en $\mathcal{NP}$.

4.2.1 La clase $\mathcal{P}$ (Polinómicamente acotado)

Decimos que un algoritmo está polinómicamente acotado si su complejidad de peor caso está acotada por una función polinómica del tamaño de las entradas (es decir, si existe un polinomio p tal que, para toda entrada de tamaño n , el algoritmo termine después de cuando más $p(n)$ pasos).

Decimos que un problema está polinómicamente acotado si existe un algoritmo polinómicamente acotado para resolverlo.

$\mathcal{P}$ es la clase de problemas de decisión que están polinómicamente acotados.

Podría parecer un tanto extravagante utilizar la existencia de una cota de tiempo polinómica como criterio para definir la clase de problemas más o menos razonables: los polinomios pueden ser muy grandes. No obstante, hay varias razones de peso para hacerlo.

Si bien no es verdad que todos los problemas en $\mathcal{P}$ tengan un algoritmo de eficiencia aceptable, sí podemos asegurar que, si un problema no está en $\mathcal{P}$, será extremadamente costoso y probablemente imposible de resolver en la práctica.

Un motivo para usar una cota polinómica para definir $\mathcal{P}$ es que los polinomios tienen propiedades de “cierre”. Podríamos obtener un algoritmo para un problema complejo combinando varios algoritmos para problemas más sencillos. La complejidad del algoritmo compuesto podría estar acotada por la suma, multiplicación y composición de las complejidades de sus algoritmos componentes.

Puesto que los polinomios están cerrados bajo estas operaciones, cualquier algoritmo que se construya de diversas formas naturales a partir de varios algoritmos polinómicamente acotados también estará polinómicamente acotado.

Un motivo más para emplear una cota polinómica es que hace a $\mathcal{P}$ independiente del modelo de cómputo formal específico que se use.

Los modelos difieren en cuanto al tipo de operaciones permitidas, los recursos de memoria disponibles y los costos asignados a diferentes operaciones.

Un problema que requiere $\Theta(f(n))$ pasos con un modelo podría requerir más de $\Theta(f(n))$ pasos con otro, pero en el caso de prácticamente todos los modelos realistas, si un problema está acotado polinómicamente con uno, estará acotado polinómicamente con los demás.

4.2.2 La clase $\mathcal{NP}$ (No determinista Polinómicamente acotado)

Es el conjunto de problemas que pueden ser resueltos en tiempo polinómico por una máquina de Turing no determinista.

$\mathcal{NP}$ es la clase de problemas de decisión para la cual existe un algoritmo no determinista polinómicamente acotado.

$\mathcal{NP}$ es la clase de problemas de decisión para los que una solución propuesta dada con una entrada dada se puede verificar rápidamente (en tiempo polinómico) para determinar si realmente es una solución (es decir, si satisface los requisitos del problema)

La importancia de esta clase de problemas de decisión es que contiene muchos problemas de búsqueda y de optimización para los que se desea saber si existe una cierta solución o si existe una mejor solución que las conocidas.

En esta clase están el problema del agente viajero donde se quiere saber si existe una ruta óptima que pasa por todos los nodos en un cierto grafo y el problema de satisfacibilidad booleana en donde se desea saber si una cierta fórmula de lógica proposicional puede ser cierta para algún conjunto de valores booleanos para las variables.

Dada su importancia, se han hecho muchos esfuerzos para encontrar algoritmos que decidan algún problema de $\mathcal{NP}$ en tiempo polinómico. Sin embargo, pareciera que para algunos problemas de $\mathcal{NP}$ (los del conjunto $\mathcal{NP} - \text{completo}$) no es posible encontrar un algoritmo mejor que simplemente realizar una búsqueda exhaustiva.

Los nombres de familias de problemas con las siglas $\mathcal{NP}$ es algo confusa. Los problemas $\mathcal{NP} - \text{hard}$ no son todos $\mathcal{NP}$, a pesar de que estas siglas aparecen es el nombre de la familia. Sin embargo, los nombres están actualmente muy arraigados y plantear un cambio de nomenclatura resulta poco realista. Por otra parte, las familias de problemas con las siglas $\mathcal{NP}$ son todas definidas tomando como referencia la familia $\mathcal{NP}$

4.3 Reducción Polinómica y la Clase $\mathcal{NP} - \text{completa}$

Esta es la definición formal de “no es más difícil que” que necesitamos.

Definición 4.3.1. Si $P, Q \in \mathcal{NP}$, y si una instancia de P puede convertirse en tiempo polinómico en una instancia de Q , entonces P es polinómicamente reducible a Q .

Esto significa que si tenemos un algoritmo para el problema Q , se puede utilizar para resolver el problema P con un sobre coste que es polinómico en el tamaño de la instancia.

Ahora definimos la clase de problemas “más difíciles”.

Definición 4.3.2. $\mathcal{NP}$ – completo, la clase de problemas $\mathcal{NP}$ – completos, es el subconjunto de problemas $P \in \mathcal{NP}$ tal que para todos $Q \in \mathcal{NP}$, Q es polinómicamente reducible a P .

Es un hecho notable no solo que $\mathcal{NP}$ – completo no está vacío, sino que todos los problemas de decisión en nuestra segunda lista están en $\mathcal{NP}$ – completo. Entonces, ¿cómo se puede demostrar que un problema está en $\mathcal{NP}$ – completo?

El paso más importante es demostrar que $\mathcal{NP}$ – completo no está vacío.

Escrito como un programa entero 0-1, una instancia de SATISFACCIÓN es el problema de decisión: Dado $N = \{1, \dots, n\}$, y $2m$ subconjuntos $\{C_i\}_{i=1}^m$ y $\{D_i\}_{i=1}^m$ de N ¿tiene el programa entero 0-1:

$$\sum_{j \in C_i} x_j + \sum_{j \in D_i} (1 - x_j) \geq 1 \text{ para } i = 1, \dots, m$$

$$x \in B^n$$

una solución factible?

Es obvio que SATISFACCIÓN está en $\mathcal{NP}$. Cook demostró en 1970 que SATISFACCIÓN está en $\mathcal{NP}$ – completo.

Ahora, indicamos cómo se puede utilizar el lema de reducción para demostrar que todos los problemas de la segunda lista y muchos otros están en $\mathcal{NP}$ – completo.

Ahora, restablecemos el Lema de Reducción (Proposición 4.1.1) de manera más formal.

Proposición 4.3.1. Supongamos que los problemas $P, Q \in \mathcal{NP}$.

- (i) Si $Q \in \mathcal{P}$ y P es polinómicamente reducible a Q , entonces $P \in \mathcal{P}$.
- (ii) Si $P \in \mathcal{NP}$ – completo y P es polinómicamente reducible a Q , entonces $Q \in \mathcal{NP}$ – completo.

Prueba. (ii) Consideremos cualquier problema $R \in \mathcal{NP}$. Como $P \in \mathcal{NP}$, R es reducible en tiempo polinómico a P . Sin embargo, P es reducible en tiempo polinómico a Q por hipótesis, y por lo tanto, R es reducible en tiempo polinómico a Q . Como esto se cumple para todos los $R \in \mathcal{NP}$, $Q \in \mathcal{NP}$ – completo.

Corolario 4.3.1. *Si $\mathcal{P} \cap \mathcal{NP} \neq \emptyset$, entonces $\mathcal{P} = \mathcal{NP}$.*

Prueba. Supongamos que $Q \in \mathcal{P} \cap \mathcal{NP}$ y tomemos $R \in \mathcal{NP}$. Como $R \in \mathcal{NP}$ y $Q \in \mathcal{NP}$, R es reducible en tiempo polinómico a Q . Por (i), como $Q \in \mathcal{P}$ y R es reducible en tiempo polinómico a Q , $R \in \mathcal{P}$. Así que $\mathcal{NP} \subseteq \mathcal{P}$ y por lo tanto $\mathcal{P} = \mathcal{NP}$.

La lista de problemas conocidos que están en $\mathcal{NP}$ es ahora enorme. Algunos de los problemas más básicos en $\mathcal{NP}$ son los problemas de nuestra segunda lista.

4.4 El problema $\mathcal{P} = \mathcal{NP}$

El problema $\mathcal{P}$ vs $\mathcal{NP}$ es determinar si cada lenguaje aceptado por algún algoritmo no determinista en tiempo polinomial también es aceptado por algún algoritmo (determinista) en tiempo polinómico. Para determinar el problema es necesario dar exactamente el modelo formal del ordenador. El modelo informático estándar en la teoría de la computabilidad es la máquina de Turing, introducida por Alan Turing en 1936. Aunque el modelo fue introducido antes de que las computadoras físicas fueran construidas, sin embargo continúa siendo aceptado como el modelo de computadora apropiado con el propósito de definir la noción de función computable.

Informalmente la clase $\mathcal{P}$ es la clase de problemas de decisión solucionables por algún algoritmo dentro de un número de pasos limitados por algún polinomio fijo en la longitud de la entrada. Turing no estaba preocupado por la eficiencia de sus máquinas, sino que su preocupación era si pueden simular algoritmos arbitrarios con suficiente tiempo. Resulta, sin embargo, que las máquinas de Turing generalmente pueden simular modelos de computadora más eficientes (por ejemplo, máquinas equipadas con muchas cintas o una memoria de acceso aleatorio ilimitada) al cuadrar o cubicar el tiempo de cálculo. Así $\mathcal{P}$ es una clase robusta y tiene definiciones equivalentes sobre una clase grande de modelos de computadora. Aquí seguimos la práctica estándar y definimos la clase $\mathcal{P}$ en términos de máquinas Turing. Formalmente los elementos de la clase $\mathcal{P}$ son idiomas. Sea Σ un alfabeto finito (es decir, un conjunto finito no vacío) con al menos dos elementos, y sea Σ^* el conjunto de cadenas finitas sobre Σ . Entonces un lenguaje sobre Σ es un subconjunto L de Σ^* . Cada máquina de Turing M tiene un alfabeto de entrada asociado Σ . Para cada cadena w en Σ^* hay un cálculo asociado con M con entrada w . (Las nociones de máquina de Turing y cómputo se definen formalmente en el apéndice.) Decimos que M acepta w si este cálculo termina en el estado de aceptación. Tenga en cuenta que M no puede aceptar w ya sea si este cálculo termina en el estado de rechazo, o si el cálculo no termina. El lenguaje aceptado por M , denotado $L(M)$, tiene asociado el alfabeto Σ y se

define por:

$$L(M) = \{w \in \Sigma^* \mid M \text{ acepta } w\}$$

Denotamos por $t_M(w)$ el número de pasos en el cálculo de M en la entrada w . Si este cálculo nunca se detiene, entonces $t_M(w) = \infty$. Para $n \in \mathbb{N}$ denotamos por $T_M(n)$ el peor tiempo de ejecución de M ; es decir,

$$TM(n) = \max\{t_M(w) \mid w \in \Sigma^n\}$$

donde Σ_n es el conjunto de todas las cadenas sobre Σ de longitud n . Decimos que M se ejecuta en tiempo polinomial si existe k tal que para todos n , $T_M(n) \leq n^k + k$. Ahora definimos la clase $\mathcal{P}$ de lenguajes por $\mathcal{P} = \{L \mid L = L(M) \text{ para algunas máquinas de Turing } M \text{ que se ejecuta en tiempo polinomial}\}$.

La notación $\mathcal{NP}$ significa “*tiempo polinomial no determinista*”, ya que originalmente $\mathcal{NP}$ se definió en términos de máquinas no deterministas (es decir, máquinas que tienen más de un posible movimiento de una configuración dada). Sin embargo, ahora es costumbre dar una definición equivalente usando la noción de una relación de comprobación, que es simplemente una relación binaria $R \subseteq \Sigma^* \times \Sigma_1^*$ para algunos alfabetos finitos Σ y Σ_1 . Asociamos con cada relación R un lenguaje L_R sobre $\Sigma \cup \Sigma_1 \cup \{\#\}$ definido por

$$L_R = \{w\#y \mid R(w, y)\}$$

Donde el símbolo $\#$ no está en Σ . Decimos que R es *tiempo polinomial* si y solo si $L_R \in \mathcal{P}$. Ahora definimos la clase $\mathcal{NP}$ de lenguajes por la condición de que un lenguaje L sobre Σ está en $\mathcal{NP}$ si y solo si hay $k \in \mathbb{N}$ y una relación de verificación de tiempo polinomial R tal que para todos $w \in \Sigma$,

$$(w \in L \Leftrightarrow \exists y(|y| \leq |w|^k \text{ y } R(w, y)))$$

donde $|w|$ y $|y|$ denotan las longitudes de w y y , respectivamente.

4.4.1 Planteamiento del problema

$\mathcal{P} = \mathcal{NP}$? Es fácil ver que la respuesta es independiente del tamaño del alfabeto Σ (asumimos) $|\Sigma| \geq 2$, ya que las cadenas sobre un alfabeto de cualquier tamaño fijo pueden ser codificadas eficientemente por cadenas sobre un alfabeto binario. (Para $|\Sigma| = 1$ el problema

sigue abierto, aunque es posible que $\mathcal{P} = \mathcal{NP}$ en este caso pero no en el caso general.) Es trivial mostrar que $\mathcal{P} \subseteq \mathcal{NP}$, ya que para cada lengua L sobre Σ , si $L \in \mathcal{P}$ entonces podemos definir la relación de comprobación de tiempo polinomial $R \subseteq \Sigma^* \cup \Sigma^*$ por

$$R(w, y) \Leftrightarrow w \in L.$$

para todo $w, y \in \Sigma^*$.

Aquí hay dos ejemplos simples, usando notación decimal para codificar números naturales: El conjunto de cuadrados perfectos está en $\mathcal{P}$, ya que el método de Newton se puede usar para aproximar eficientemente las raíces cuadradas. El conjunto de números compuestos está en $\mathcal{NP}$, donde (denotando la notación decimal para un número natural c por $\bar{c}$) la relación de comprobación de tiempo polinómico asociado R está dada por

$$R(a, b) \Leftrightarrow 1 < b < a \text{ y } b|a$$

4.5 Consecuencias de $\mathcal{P} = \mathcal{NP}$ o $\mathcal{P} \neq \mathcal{NP}$

La mayoría de los problemas de interés han sido mostrados como pertenecientes a $\mathcal{P}$ o $\mathcal{NP}^{\mathcal{C}}$. Además, nadie ha logrado demostrar que $\mathcal{P} = \mathcal{NP}$ ni mostrar que $\mathcal{P} \neq \mathcal{NP}$. Sin embargo, dado el enorme número de problemas en $\mathcal{NP}^{\mathcal{C}}$ para los cuales no se ha encontrado un algoritmo polinómico, es una hipótesis de trabajo práctica que $\mathcal{P} \neq \mathcal{NP}$.

Entonces, ¿cómo debemos interpretar los resultados y observaciones anteriores?

Una primera observación importante concierne a la clase $\mathcal{NP}$. Típicamente, los problemas en esta clase tienen un conjunto de soluciones factibles muy grande (exponencialmente grande), y estos problemas pueden, en teoría, resolverse enumerando las soluciones factibles.

Un pesimista podría decir que, dado que la mayoría de los problemas parecen ser difíciles (es decir, su versión de decisión pertenece a $\mathcal{NP}^{\mathcal{C}}$), no tenemos esperanza de resolver instancias de gran tamaño (porque en el peor de los casos no podemos esperar hacerlo mejor que mediante la enumeración) y por lo tanto deberíamos rendirnos.

Un matemático (valiente) podría proponerse hacerse famoso demostrando que $\mathcal{P} \neq \mathcal{NP}$.

Un matemático (reflexivo) podría decidir hacer una pregunta diferente: ¿Puedo encontrar un algoritmo que esté garantizado para encontrar una solución “cercana a la óptima” en tiempo polinómico en todos los casos?

Un probabilista (reflexivo) podría también hacer una pregunta diferente: ¿Puedo encontrar un algoritmo que se ejecute en tiempo polinómico con alta probabilidad y que esté garantizado para encontrar una solución óptima o “cercana a la óptima” con alta probabilidad?

Un ingeniero comenzaría a buscar un algoritmo heurístico que produzca soluciones prácticamente utilizables.

Tu jefe podría decir: No me importa la teoría de programación entera. Tú solo preocúpate por nuestro problema de programación. Dame un horario de producción factible para mañana en el que el pedido de John Brown and Daughters esté listo para salir antes de las 4 p.m.

Un profesor con dificultades podría decir: Genial. Anteriormente, estaba tratando de desarrollar un algoritmo para resolver todos los programas enteros, y publicando un artículo cada dos años explicando por qué no tenía éxito. Ahora, sé que bien podría estudiar cada problema $\mathcal{NP}$ individualmente. Como hay miles de ellos, debería poder escribir 20 artículos al año.

No hace falta decir que casi todos tienen razón. No hay una solución fácil y rápida, pero los problemas no desaparecerán, y cada vez más problemas prácticos fascinantes e importantes se están formulando como programas enteros. Así que, a pesar de la teoría de completitud de $\mathcal{NPC}$, utilizando una combinación adecuada de teoría, algoritmos, experiencia y cálculo intensivo, se pueden y se deben encontrar soluciones verificablemente buenas para instancias grandes.

Definición 4.5.1. *Un problema de optimización para el cual el problema de decisión pertenece a $\mathcal{NPC}$ se llama $\mathcal{NP}$ -duro.*

La complejidad algorítmica es un pilar central en el estudio de la programación entera. Su inclusión en la literatura académica y técnica proporciona a los lectores las herramientas necesarias para comprender, evaluar y abordar problemas complejos de manera efectiva y eficiente. Este conocimiento no solo ilumina las dificultades inherentes a estos problemas, sino que también abre caminos hacia soluciones innovadoras y prácticas en el campo de la programación entera. Además los problemas enteros son $\mathcal{NP}$ – duros.

Por último concluir que el Clay Mathematics Institute de Cambridge ha incluido al problema $\mathcal{P}$ versus $\mathcal{NP}$ como uno de los siete problemas del Premio Millennium y ha anunciado que, si alguien lo resuelve, ganaría un millón de dólares.

Capítulo 5

Implementación en R

En este capítulo vamos a resolver un problema de programación lineal entera a partir de varios métodos. Tenemos el siguiente problema:

$$\begin{aligned} \text{Maximizar: } & x_1 + 5x_2 - x_3 \\ \text{Sujeto a: } & x_1 + x_2 + x_3 \geq 100 \\ & 5x_1 - x_2 + 8x_3 \leq 500 \\ & -x_1 + x_2 + 2x_3 = 0 \\ & 2x_1 + x_2 + 12x_3 \leq 800 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

5.1 Función `simplex()` del package `boot`

Sirve para problemas de tamaño no muy grande. Instalar y cargar package previamente.

- Función: `simplex(a, A1, b1, A2, b2, A3, b3, maxi, n.iter, eps)`
- Argumentos:
- `a`: coeficientes de la función objetivo
- `A1`: matriz de restricciones de tipo $\leq$ (sólo la parte de coeficientes)

- b1: vector de términos independientes de las restricciones de tipo $\leq$ (correspondientes a la matriz A1)
- A2: idem con restricciones de tipo $\geq$
- b2: idem correspondientes a la matriz A2
- A3: idem con restricciones de tipo $=$
- b3: idem correspondientes a la matriz A3
- maxi: Poner TRUE si se desea maximizar la función objetivo
- n.iter: número máximo de iteraciones en cada fase (usa método de dos fases)
- eps: tolerancia de coma flotante
- Devuelve: un objeto de la clase simplex, que es una lista con componentes (entre otras):
- \$soln: solución óptima, si se alcanza
- \$solved: código de terminación:
 - 1 indica solución óptima encontrada
 - 0 indica que se ha alcanzado el número máximo de iteraciones sin acabar
 - -1 indica que no existe solución factible
- \$value: valor de la función objetivo en la solución \$soln
- \$val.aux: valor de las variables artificiales en el óptimo
- etc.

```
library(boot)

# Definir los datos del problema
a <- c(1, 5, -1)
A1 <- matrix(c(5, -1, 8), c(2, 1, 12), nrow=2, ncol=3)
b1 <- c(500, 800)
A2 <- matrix(c(1, 1, 1), nrow = 1)
b2 <- 100
A3 <- matrix(c(-1, 1, 2), nrow=1)
b3 <- 0
maxi <- TRUE
```



```
n.iter <- 1000000
eps <- 1e-6

# Resolver el problema
resultado <- simplex(a, A1, b1, A2, b2, A3, b3, maxi, n.iter, eps)

# Mostrar el resultado
print(resultado)

##
## Linear Programming Results
##
## Call : simplex(a = a, A1 = A1, b1 = b1, A2 = A2, b2 = b2, A3 = A3, b3 = b3,
##      maxi = maxi, n.iter = n.iter, eps = eps)
##
## Maximization Problem with Objective Function Coefficients
## x1 x2 x3
##  1  5 -1
##
##
## Optimal solution has the following values
##  x1  x2  x3
## 125 125   0
## The optimal value of the objective function is 750.
```

La solución óptima para el problema de programación lineal proporcionado es:

$$x_1 = 125$$

$$x_2 = 125$$

$$x_3 = 0$$

Esto significa que para maximizar la función objetivo, se debe asignar $x_1 = 125$, $x_2 = 125$ y $x_3 = 0$. Con estas asignaciones, el valor óptimo de la función objetivo es 750

5.2 Función `lp()` del package `lpSolve`

Se trata de otra función para resolver PPLs, pero más completa pues admite variables enteras (programación entera) y variables binarias (programación 0-1). Para usarla hay que instalar y cargar el package previamente.

- Función: `lp` (`direction`, `objective.in`, `const.mat`, `const.dir`, `const.rhs`, `int.vec`, `binary.vec`)
- Argumentos:
 - `direction`: poner 'max' si se quiere maximizar la función objetivo
 - `objective.in`: vector con coeficientes de la función objetivo
 - `const.mat`: matriz de coeficientes de las restricciones
 - `const.dir`: vector de cadenas de texto indicando la dirección de cada restricción (" $\leq$ ", " $=$ ", o " $\geq$ ")
 - `const.rhs`: vector de términos independientes de las restricciones
 - `int.vec`: Vector indicando qué variables son enteras
 - `binary.vec`: Vector indicando qué variables son 0-1
- Devuelve: un objeto de la clase `lp`, que es una lista con componentes (entre otras):
 - `$objval`: valor de la función objetivo en el óptimo
 - `$solution`: solución óptima
 - `status`: indicador numérico:
 - 0: éxito
 - 2: no hay solución factible

Vamos a resolver el mismo problema de antes pero empleando la función `lp()`.

```
library(lpSolve)

# Definir los datos del problema
direction <- "max" #
objective.in <- c(1, 5, -1)
```

```

const.mat <- rbind(c(1, 1, 1), c(5, -1, 8), c(-1, 1, 2), c(2, 1, 12))
const.dir <- c(">=", "<=", "=", "<=")
const.rhs <- c(100, 500, 0, 800)
int.vec <- 1:3
binary.vec <- numeric(0)

# Resolver el problema
resultado <- lp(direction = direction,
               objective.in = objective.in,
               const.mat = const.mat,
               const.dir = const.dir,
               const.rhs = const.rhs,
               int.vec = int.vec,
               binary.vec = binary.vec)

# Mostrar el resultado
resultado

```

```
## Success: the objective function is 750
```

```
resultado$solution
```

```
## [1] 125 125 0
```

La solución óptima para el problema de programación lineal proporcionado, con el objetivo de maximizar la función objetivo, es la siguiente:

Se asignan los siguientes valores a las variables de decisión:

$$x_1 = 125$$

$$x_2 = 125$$

$$x_3 = 0$$

Con estas asignaciones, se alcanza un valor óptimo de la función objetivo de 750.

5.3 Función `Rglpk_solve_LP()` de la librería `Rglpk`

Usando `Rglpk_solve_LP` podemos resolver fácilmente problemas como el siguiente:

```
library(Rglpk)
```

```
## Loading required package: slam
```

```
## Using the GLPK callable library version 5.0
```

```
obj <- c(1, 5, -1)
mat <- matrix(c(1, 1, 1,
               5, -1, 8,
               -1, 1, 2,
               2, 1, 1), nrow = 4, byrow = TRUE)
dir <- c(">=", "<=", "==", "<=")
rhs <- c(100, 500, 0, 800)
max <- TRUE

result <- Rglpk_solve_LP(obj, mat, dir, rhs, max = max)

print(result)
```

```
## $optimum
## [1] 750
##
## $solution
## [1] 125 125 0
##
## $status
## [1] 0
##
## $solution_dual
## [1] 0 0 -26
##
## $auxiliary
```

```
## $auxiliary$primal
## [1] 250 500    0 375
##
## $auxiliary$dual
## [1] 0.0 1.5 6.5 0.0
##
##
## $sensitivity_report
## [1] NA
```

En este orden, se define la función objetivo mediante un vector (`obj`) que contiene los pesos de cada variable; después se define la matriz (`mat`) que contiene la parte izquierda de las inecuaciones que describen las restricciones; seguidamente, se describe la dirección (`dir`) de cada inecuación; a continuación, la parte derecha de las inecuaciones (`rhs`), y, finalmente, se indica que se trata de un problema de maximización (`max`). Para acabar, se llama a la función que hace de interfaz para resolver el problema mediante la librería `Rglpk` para programación lineal.

La solución óptima maximiza la función objetivo con un valor de 750, asignando 125 a x_1 y x_2 y 0 a x_3 . La solución es factible y óptima sin problemas señalados. Los valores duales proporcionan información sobre la sensibilidad de la solución a cambios en las restricciones, destacando un impacto significativo en la tercera restricción.

Conclusiones

El trabajo sobre programación lineal entera (PLE) subraya su importancia para resolver problemas complejos en áreas como la logística y la producción. La PLE permite manejar variables enteras, lo cual es esencial para situaciones reales con restricciones específicas. Se han explorado y comparado diversos métodos de resolución, como el método del simplex, el algoritmo de ramificación y acotación y los algoritmos de corte, cada uno adecuado para diferentes tipos de problemas.

La investigación también aborda la complejidad algorítmica, discutiendo las clases de problemas $\mathcal{P}$ y $\mathcal{NP}$ y la relevancia del problema $\mathcal{P} = \mathcal{NP}$. Esta discusión es crucial para entender la dificultad inherente de los problemas de PLE y su impacto en la optimización y la teoría de la computación.

La implementación práctica de algoritmos en el lenguaje R demuestra la aplicabilidad y efectividad de estas técnicas en situaciones reales. El uso de librerías específicas en R permite resolver problemas de PLE de manera eficiente y accesible.

En resumen, la programación lineal entera es una herramienta poderosa y versátil para la optimización de recursos y la toma de decisiones en diversos campos. Los métodos y herramientas discutidos proporcionan una base sólida tanto teórica como práctica para abordar problemas complejos de manera efectiva.

Bibliografía

Arreola Risa, J. S., & Arreola Risa, A. (2003). Programación lineal: Una introducción a la toma de decisiones cuantitativa. Paraninfo.

Bazaraa, M., Jarvis, J. J., & Sherali, H. D. (1981). Programación lineal y flujo en redes.

Taha, H. A. (2014). Integer programming: theory, applications, and computations. Academic Press.

Pardo Llorente, L., Pardo LLorente, J. A., & Ortega, A. F. (1989). Programación lineal entera:: aplicaciones prácticas en la empresa.

Hu, T. C. (1969). Integer programming and network flows (Vol. 45701). Reading: Addison-Wesley.

<https://www.uv.es/~sala/Clase14.pdf>

https://ocw.ehu.eus/pluginfile.php/40409/mod_resource/content/1/software_mate/material-de-estudio-1/tema-11-investigacion-operativa-con-r.pdf

<https://www3.uji.es/~gregori/docencia/mt1021-1617/mt1021-1617-la-5-proglin.html>

<http://algoritmosandmore.blogspot.com/p/p-y-np.html>

Barthélemy, J. P., Cohen, G., & Lobstein, A. (1996). Algorithmic Complexity and Communication Problems. UCL Press.

<https://www.claymath.org/wp-content/uploads/2022/06/pvsnp.pdf>

Wolsey, L. (2020). Front Matter. In Integer Programming, L. Wolsey (Ed.). <https://doi.org/10.1002/9781119606475.fmatter>

Antonio Rufián Lizana, Gabriel Ruiz Garzón, Rafaela Osuna Gómez (2011). Métodos de

optimización matemática : [manual para la resolución de problemas de optimización aplicados a la toma de decisiones empresariales]

«Universidad de Sevilla». Disponible en <https://www.us.es>, Página web.

Allaire, JJ; Xie, Yihui; Dervieux, Christophe; McPherson, Jonathan; Luraschi, Javier; Ushey, Kevin; Atkins, Aron; Wickham, Hadley; Cheng, Joe; Chang, Winston y Iannone, Richard *rmarkdown: Dynamic Documents for R*, 2023. <https://github.com/rstudio/rmarkdown>. R package version 2.25.

Facultad de Matemáticas (Univ. Sevilla), s.f. <https://www.matematicas.us.es>.

Lopez, Juan Fernando; Fernández Henao, Sergio y Morales, Marcela María «Aplicación de la programación por metas en la distribución de servicios entre empresas operadoras del sistema de transporte masivo». *Scientia et technica*, 2007, 13(37), pp. 339–343.

Luque Calvo, Pedro L. *Escribir un Trabajo Fin de Estudios con R Markdown*, 2017. <http://destio.us.es/calvo>.

Cómo crear Tablas de información en R Markdown, 2019. <http://destio.us.es/calvo>.

Luque Calvo, Pedro L. «Página personal de Pedro L. Luque», 2021. <http://destio.us.es/calvo>.

R Core Team *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2016. <https://www.R-project.org/>.

RStudio Team *RStudio: Integrated Development Environment for R*. RStudio, Inc., Boston, MA, 2015. <http://www.rstudio.com/>.

Techopedia «“Definition - What does Business Intelligence (BI) mean?”». Disponible en <https://www.techopedia.com/definition/345/business-intelligence-bi>, 2017.

Xie, Yihui *knitr: A General-Purpose Package for Dynamic Report Generation in R*, 2023. <https://yihui.org/knitr/>. R package version 1.45.